

[Cover Page]

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

A. Kompetensi

Mahasiswa mampu memahami dan mengimplementasikan pengaruh parameter pengendali PID dalam mengatur rotasi pada modul *Swerve Drive*.

B. Sub Kompetensi

Setelah melaksanakan praktikum, mahasiswa diharapkan mampu:

1. Mengidentifikasi pengaruh masing-masing parameter pengendali Kp (proportional), Ki (integral), dan Kd (derivative) terhadap respons rotasi pada modul *Swerve Drive*.
2. Mengatur nilai parameter PID untuk mencapai rotasi untuk mencapai rotasi roda yang stabil dan akurat.
3. Menganalisis performa sistem kendali berdasarkan parameter *overshoot*, *rise time*, *settling time*, dan *steady-state error (SSE)*.
4. Menyimpulkan hubungan antara perubahan parameter PID dengan karakteristik dinamis sistem rotasi.

C. Dasar Teori

1. Pengertian dan Prinsip Kerja *Swerve Drive*

Swerve Drive dikenal juga sebagai *Crab Drive* atau *Coaxial Drive* adalah sistem penggerak holonomik yang memberikan mobilitas pada *Mobile Robot*. Robot disebut sebagai holonomik apabila jumlah derajat kebebasan yang dapat dikendalikan sama dengan total derajat kebebasan di ruang kerjanya. Robot di bidang 2 dimensi (2D) berarti robot dapat bergerak secara instan ke arah manapun termasuk maju, mundur, ke samping kiri, maupun kanan dan berotasi secara bersamaan, tanpa perlu melakukan manuver berbelok. Terdapat dua prinsip utama *Swerve Drive* adalah penggunaan modul-modul roda independen, tiap modul terdiri dari dua aktuator berupa motor:

- a. Motor penggerak (*Drive Motor*) untuk mengontrol kecepatan putaran roda untuk menggerakkan robot.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

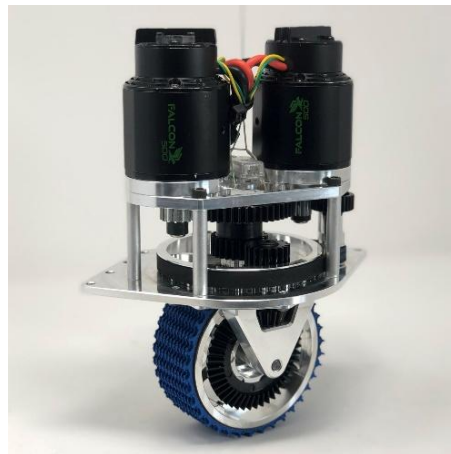
	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

- b. Motor belok (*Steering Motor*) untuk mengendalikan orientasi (sudut) roda terhadap sasis robot.

Dengan kemampuan untuk mengarahkan setiap roda secara independen, robot dapat mencapai pergerakan *omnidirectional* yang sesungguhnya.

Gambar 1. Modul Swerve Drive

(Sumber: <https://www.swervedrivespecialties.com/>)



2. PID Controllers

Kendali PID atau PID controller yang merupakan kependekan dari (Proportional-Integral-Derivative controller) digunakan sistem kendali untuk menghitung variabel kesalahan (*error*) antara nilai yang diinginkan (*setpoint*) dan variabel proses yang terukur (*process variable*). Hasil keluaran dari proses kendali PID digunakan untuk meminimalkan *error* tersebut. Pengendalian ini menggunakan tiga variabel kontrol utama antara lain: kontrol proporsional, integral, dan derivatif, yang diatur sedemikian rupa sehingga mampu meningkatkan stabilitas dan kinerja dalam proses otomasi.

Referensi atau target yang diinginkan disebut sebagai *setpoint* dan keluarannya disebut sebagai variabel proses terukur. Berikut merupakan konvensi penamaan variabel umum untuk jumlah yang relevan:

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

$$\text{setpoint} = r(t)$$

$$\text{error} = e(t)$$

$$\text{control input} = u(t)$$

$$\text{output} = y(t)$$

Variabel kesalahan atau error dapat dihitung dengan mengurangi variabel referensi dengan variabel output, sehingga $e(t) = r(t) - y(t)$.

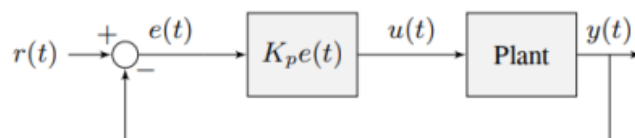
a. Proportional

Variabel kontrol ini bertujuan untuk mengurangi kesalahan posisi menjadi nol. Kesalahan posisi dihitung sebagai selisih antara nilai setpoint dan nilai yang terukur saat ini, sehingga dapat diformulasikan sebagai berikut:

$$u(t) = k_p e(t)$$

Di mana K_p adalah gain penguatan proporsional dan $e(t)$ adalah error pada waktu t .

Gambar 2 Diagram Blok Kendali Proportional



b. Integral

Variabel kontrol integral mengakumulasi kesalahan posisi dari waktu ke waktu dengan menambahkan akumulasi kesalahan ke input kendali, kontrol integral membantu mengatasi kesalahan *steady-state* yaitu kondisi di mana kesalahan di mana sistem telah mencapai kestabilan namun keluaran tidak sepenuhnya mencapai nilai yang diinginkan. Kendali integral dapat diformulasikan menjadi:

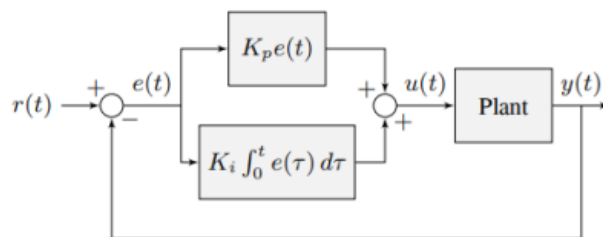
Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau$$

Di mana K_p adalah kendali penguatan proporsional, K_i adalah penguatan integral, $e(t)$ adalah eror pada waktu t , dan τ adalah variabel integrasi. $\int_0^t e(\tau) d\tau$ adalah integral dari kesalahan seiring waktu yang terakumulasi pada area di antara kurva setpoint dan keluaran.

Gambar 3 Diagram Blok Kendali Proportional-Integral



c. Derivative

Variabel kontrol derivatif bekerja dengan merespons laju perubahan kesalahan dari waktu ke waktu, kendali ini tidak hanya melihat besarnya kesalahan saat ini, tetapi juga seberapa cepat kesalahan tersebut berubah. Kendali ini bertujuan untuk mengantisipasi pergerakan sistem dan memberi *damping* agar sistem tidak mengalami *overshoot* atau osilasi berlebih, sehingga membantu meningkatkan kestabilan dan kecepatan respons sistem. Dalam sistem kendali PID, komponen derivatif berguna dalam meredam respon kendali proporsional dan integral yang terlalu agresif. Kendali derivatif dapat diformulasikan sebagai berikut:

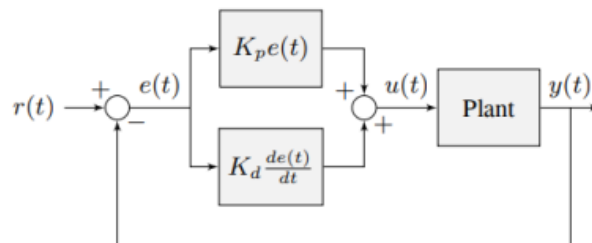
$$u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \frac{de(t)}{dt}$$

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

Di mana K_d adalah penguatan derivatif dan $\frac{de(t)}{dt}$ adalah turunan dari kesalahan terhadap waktu. Komponen ini memperkirakan arah dan kecepatan perubahan error, sehingga sistem dapat mengambil tindakan korektif sebelum kesalahan menjadi lebih besar, sehingga dapat dikatakan bahwa kendali derivatif bertindak sebagai prediktor terhadap kesalahan untuk memberikan respons kendali yang lebih halus dan stabil.

Gambar 4 Diagram Blok Kendali Proportional-Derivative



3. Analisis Respon Sistem

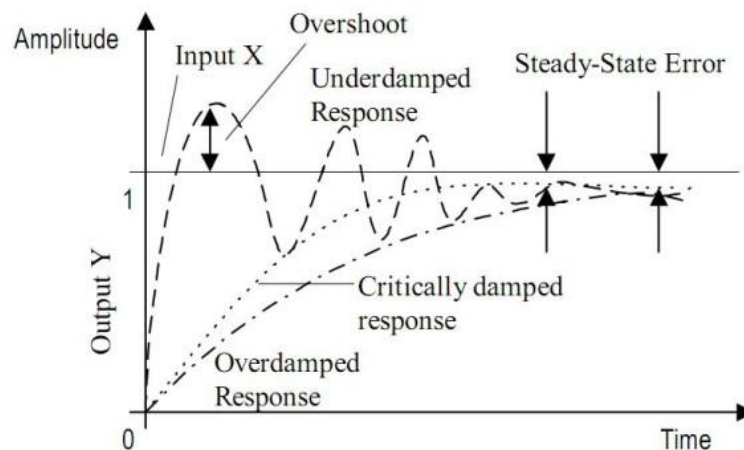
Respon sistem adalah reaksi keluaran terhadap perubahan masukan, biasanya direpresentasikan dalam bentuk kurva waktu dan menjadi dasar untuk menganalisis karakteristik dan kinerja sistem secara kualitatif dan kuantitatif. Respon sistem terbagi menjadi dua fase utama, yaitu respon transien dan respon keadaan tunak (*steady-state*). Respon transien menggambarkan bagaimana reaksi sistem sejak terjadi perubahan masukan hingga mendekati kestabilan. Sedangkan respon keadaan tunak merupakan kondisi sistem setelah waktu cukup lama ketika output sudah relatif konstan dan tidak mengalami perubahan signifikan. Karakteristik respon sistem dianalisis pada respons transien karena di fase ini terdapat informasi penting tentang dinamika sistem seperti kecepatan dan stabilitasnya.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

Gambar 5. Kurva Respon Sistem

Sumber: <https://bagaskawarasan.wordpress.com/>



Parameter dalam analisis performa respons transien sistem kendali adalah sebagai berikut:

a. Rise Time (Waktu Naik)

Rise time adalah waktu yang dibutuhkan oleh respon sistem untuk naik dari sekitar 10% ke 90% dari nilai *steady-state* (nilai akhir). Rise time diukur dengan mengambil kurva respons sistem terhadap waktu, menentukan nilai *steady-state* Y_{ss} , lalu mencari waktu saat output mencapai 10% ($0.1 \times Y_{ss}$) dan 90% ($0.9 \times Y_{ss}$) dari nilai *steady-state* tersebut. Selisih kedua waktu ini adalah nilai rise time (t_r):

$$tr = t_{90\%} - t_{10\%}$$

Jika nilai 10% dan 90% tidak langsung ditemukan dalam data, interpolasi linier antara titik data terdekat dapat dilakukan untuk memperkirakan nilai waktu tersebut.

b. Overshoot (Persentase Lonjakan Maksimum)

Overshoot mengindikasikan berapa persen nilai maksimum respon sistem melebihi *steady-state* setelah terjadi perubahan input. Overshoot dihitung sebagai:

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

$$M_p = \frac{Y_{max} - Y_{ss}}{Y_{ss}} \times 100\%$$

Di mana Y_{max} adalah nilai *peak output* tertinggi yang terjadi sebelum *output* kembali mendekati *steady-state*. *Overshoot* dinyatakan dalam persen (%) dan biasanya diukur dari grafik secara langsung dari kurva respons.

c. Settling Time (Waktu Tunak)

Settling time adalah waktu yang diperlukan untuk *output* sistem agar masuk dan tetap berada dalam batas toleransi tertentu dari nilai *steady-state*, biasanya $\pm 2\%$ atau $\pm 5\%$. Misal batas toleransi $\pm 2\%$, maka:

$$|Y(t) - Y_{ss}| \leq 0.02 \times Y_{ss}$$

$$\text{untuk semua } t \geq t_s$$

Waktu t_s ini dicatat sebagai *settling time*. Pengukuran dilakukan dengan melihat kurva dan menentukan waktu terakhir sebelum *output* stabil dan masih dalam batas toleransi tersebut.

d. Steady-State Error (SSE)

Steady-state error adalah selisih antara output sistem pada keadaan stabil dan input referensi. Dengan nilai *steady-state output* Y_{ss} dan *input setpoint* R , SSE dapat dihitung sebagai:

$$SSE = |R - Y_{ss}|$$

Dalam kasus sinyal step, jika *output* sistem tidak persis mencapai *setpoint*, nilai SSE ini menunjukkan besarnya error akhir yang tersisa.

D. Alat dan Bahan

1. Laptop atau Komputer
2. Unity 3D
3. *Software* Visual Studio Code
4. *Software* Microsoft Excel

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i> Menggunakan Metode PID	No. Job: 1
	Revisi: 00		4 x 50 menit

E. Keselamatan Kerja

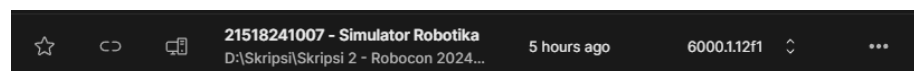
1. Selalu ikuti instruksi praktikum dari dosen, serta diskusikan jika terjadi *error* atau kendala dalam penggunaan simulator.
2. Hindari penggunaan perangkat komputer dan Unity secara berlebihan yang dapat menyebabkan kelelahan mata.
3. Gunakan perangkat komputer dengan posisi ergonomis, tinggi layar sejajar dengan pandangan dan posisi duduk yang baik untuk menghindari kelelahan saat melakukan simulasi.
4. Hindari membuka terlalu banyak aplikasi secara bersamaan agar Unity berjalan lancar dan mencegah komputer mengalami *crash*.

F. Langkah Kerja

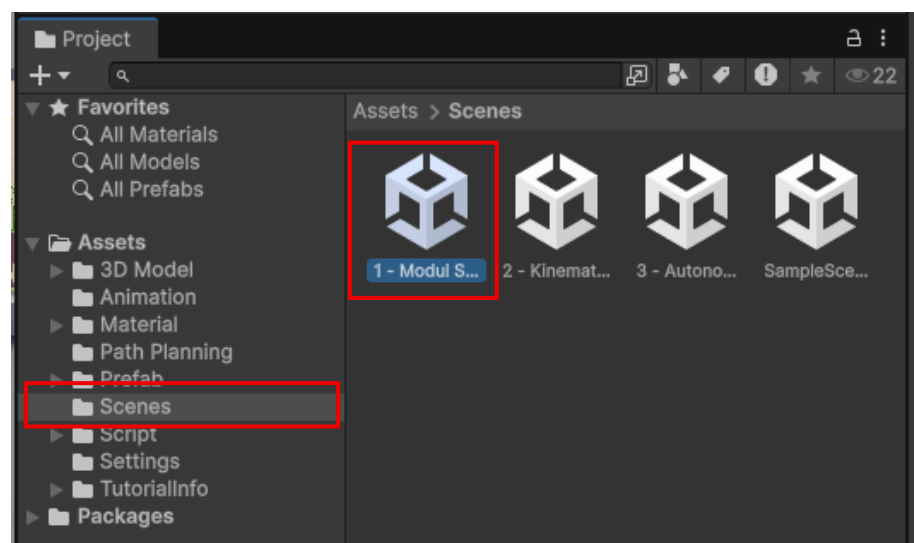
1. Buka *software* Unity Hub.



2. Buka project simulator robotika.



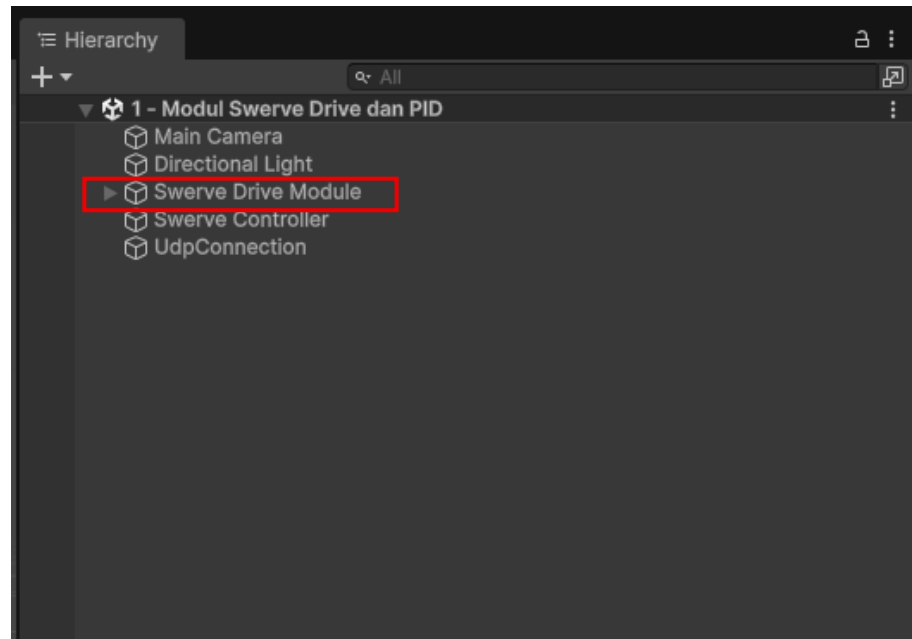
3. Pada *Assets* buka folder Scenes lalu klik dua kali pada modul pembelajaran “1 - Modul Swerve Drive dan PID”.



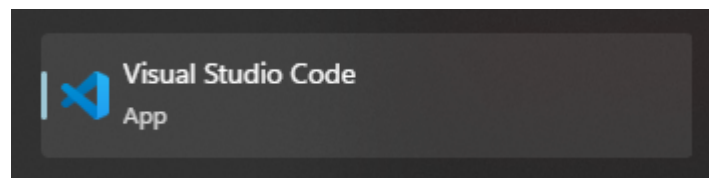
Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

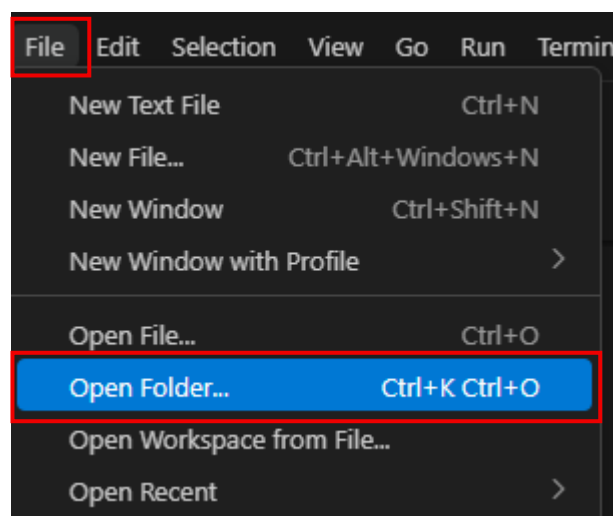
4. Klik dua kali pada *Hierarchy > Swerve Drive Module* untuk memfokuskan pandangan pada module Swerve Drive.



5. Buka *software* Visual Studio Code.



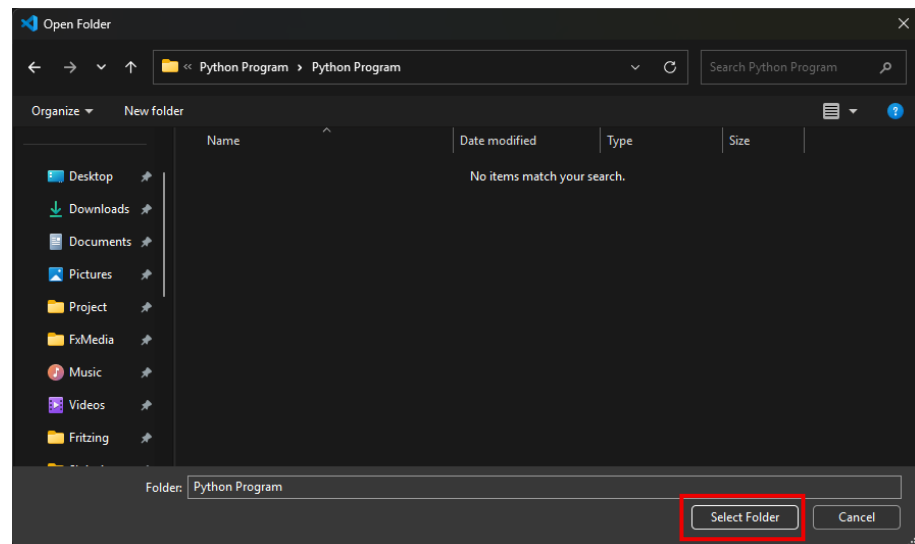
6. Cari tab *File* lalu arahkan ke *Open Folder*



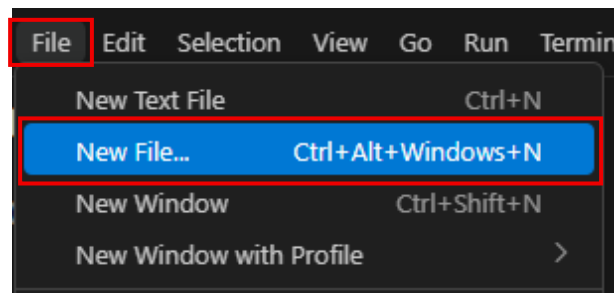
Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

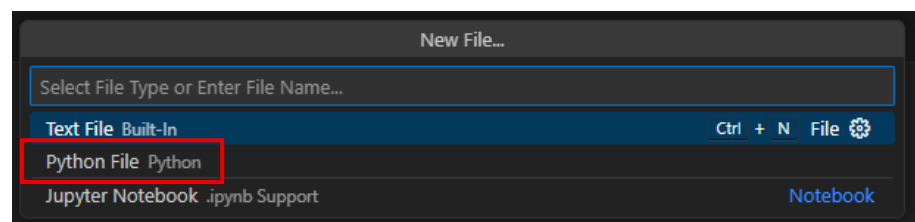
7. Cari folder untuk menyimpan program Python. Setelah menemukan tempat penyimpanan program, langkah selanjutnya klik pada Select Folder.



8. Buat file program Python baru dengan menuju ke menu *File > New File*.



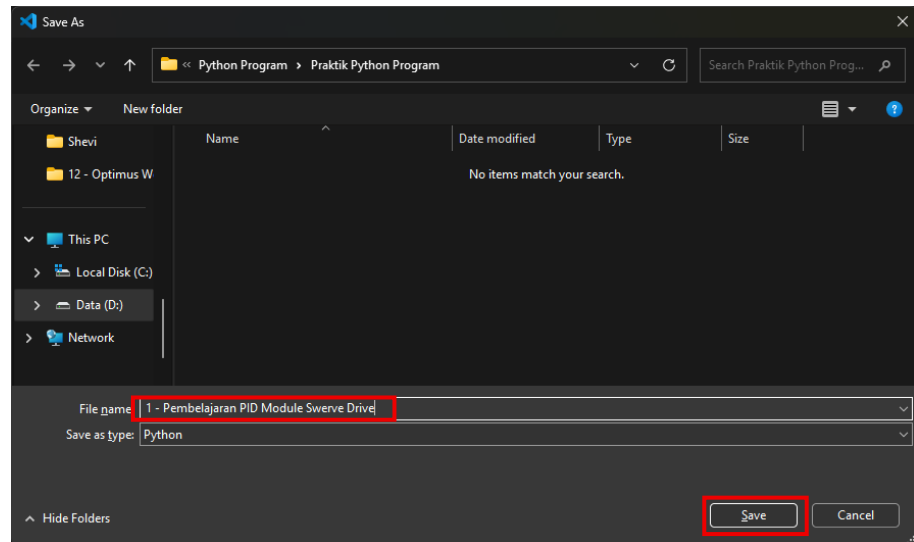
9. Pilih Python File pada dialog.



10. Tekan bersamaan kombinasi keyboard shortcut (CTRL + S) untuk menyimpan file, berikan nama file “1 - Pembelajaran PID Module Swerve Drive” lalu klik tombol Save.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit



11. Ketik program berikut pada file yang telah dibuat sebelumnya.

```
import socket
import threading
import queue
import time
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from collections import deque
import pandas as pd
import matplotlib.animation as animation
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg, NavigationToolbar2Tk

# --- Konfigurasi dipindahkan ke dalam kelas atau sebagai
konstanta ---
PYTHON_IP = '127.0.0.1'
PYTHON_PORT_LISTEN = 5005
UNITY_IP = '127.0.0.1'
UNITY_PORT_SEND = 5006
MAX_DATA_POINTS_PLOT = 200
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```
# --- Listener Function (Lebih baik sebagai metode statis
atau di luar kelas) ---
# Tidak ada perubahan besar di sini, sudah cukup baik.
def udp_listener_thread(sock, q, stop_event):
    """Fungsi ini berjalan di thread terpisah untuk
mendengarkan data UDP."""
    print("UDP listener thread dimulai.")
    while not stop_event.is_set():
        try:
            sock.settimeout(1.0)
            data_bytes, _ = sock.recvfrom(1024)
            data_str = data_bytes.decode('utf-8')
            q.put((time.time(), float(data_str)))
        except socket.timeout:
            continue
        except (ValueError, UnicodeDecodeError):
            print("Peringatan: Menerima data UDP tidak
valid.")
            continue
        except Exception as e:
            if not stop_event.is_set():
                print(f"Error kritis di UDP listener:
{e}")
            break
    print("UDP listener thread dihentikan.")

class PIDControllerApp:
    def __init__(self, master):
        self.master = master
        self.master.title("PID Controller & UDP Plotter")
        self.master.geometry("850x700")
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

# --- Inisialisasi Atribut Kelas (Enkapsulasi) ---
-

self.sock = self._setup_socket()
self.data_queue = queue.Queue()
self.stop_event = threading.Event()

# Data untuk plot dan rekaman
self.plot_data =
deque(maxlen=MAX_DATA_POINTS_PLOT)
self.session_recorder = []
self.session_pid_params = {}

# State aplikasi
self.is_running = False
self.start_time = 0
self.current_setpoint = 180.0

# Panggil metode untuk membangun UI dan memulai
listener
self._create_widgets()
self._start_listener_thread()
self.master.protocol("WM_DELETE_WINDOW",
self._on_closing)

def _setup_socket(self):
    """Membungkus setup socket dalam satu metode."""
    try:
        sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        sock.bind((PYTHON_IP, PYTHON_PORT_LISTEN))
        return sock

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

except OSError as e:
    messagebox.showerror("Socket Error", f"Gagal
bind ke port {PYTHON_PORT_LISTEN}.\nPastikan tidak ada
aplikasi lain yang menggunakan port ini.\n\nError: {e}")
    self.master.destroy()
    raise

def _create_widgets(self):
    """Membangun semua elemen GUI dalam satu metode
terpusat."""
    # --- Frame Kontrol (Kiri) ---
    control_frame = ttk.Frame(self.master,
padding="10")
    control_frame.pack(side=tk.LEFT, fill=tk.Y,
padx=10, pady=10)

    ttk.Label(control_frame, text="Kontrol PID",
font=("Helvetica", 16, "bold")).pack(pady=(0, 10))

    # Variabel Tkinter
    self.kp_var = tk.StringVar(value="1.0")
    self.ki_var = tk.StringVar(value="0.1")
    self.kd_var = tk.StringVar(value="0.05")
    self.setpoint_var =
tk.StringVar(value=str(self.current_setpoint))

    # Membuat input fields menggunakan metode helper
    self._create_entry(control_frame, "Kp:",
self.kp_var)
    self._create_entry(control_frame, "Ki:",
self.ki_var)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        self._create_entry(control_frame, "Kd:",
self.kd_var)

        self._create_entry(control_frame, "Setpoint:",
self.setpoint_var)

        ttk.Button(control_frame, text="Apply PID
Settings", command=self.apply_pid_settings).pack(pady=10,
ipady=5, fill=tk.X)

        self.start_stop_button =
ttk.Button(control_frame, text="Start PID",
command=self.toggle_pid_control)

        self.start_stop_button.pack(pady=10, ipady=8,
fill=tk.X)

        self.save_button = ttk.Button(control_frame,
text="Save Session to Excel", command=self.save_to_excel,
state=tk.DISABLED)

        self.save_button.pack(pady=10, ipady=5,
fill=tk.X)

        self.status_label = ttk.Label(control_frame,
text="Status: Ready", foreground="blue", wraplength=200)
# wraplength untuk pesan error panjang
        self.status_label.pack(side=tk.BOTTOM, pady=10)

        # --- Frame Plot (Kanan) ---
        plot_frame = ttk.Frame(self.master)
        plot_frame.pack(side=tk.RIGHT, fill=tk.BOTH,
expand=True, padx=(0,10), pady=10)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        self.fig, self.ax =
plt.subplots(facecolor='#f0f0f0') # Samakan warna
background

        self.ax.set_facecolor('ffffff')

        self.canvas = FigureCanvasTkAgg(self.fig,
master=plot_frame)

        toolbar = NavigationToolbar2Tk(self.canvas,
plot_frame, pack_toolbar=False)
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.canvas.get_tk_widget().pack(side=tk.TOP,
fill=tk.BOTH, expand=True)

        # Simpan referensi animasi untuk mencegah garbage
collection
        self._animation =
animation.FuncAnimation(self.fig, self._update_plot,
interval=50, blit=False)

        def _create_entry(self, parent, label_text,
text_variable):
            """Metode helper untuk membuat label dan
entry."""
            frame = ttk.Frame(parent)
            frame.pack(fill=tk.X, pady=5)
            ttk.Label(frame, text=label_text,
width=10).pack(side=tk.LEFT, padx=5)
            ttk.Entry(frame, textvariable=text_variable,
width=15).pack(side=tk.LEFT, expand=True, fill=tk.X)

        def _start_listener_thread(self):

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        self.listener_thread =
threading.Thread(target=udp_listener_thread,
args=(self.sock, self.data_queue, self.stop_event))
        self.listener_thread.daemon = True
        self.listener_thread.start()

# --- Logika Aplikasi ---
def toggle_pid_control(self):
    if self.is_running:
        # Proses STOP
        self._send_command_to_unity("stop_pid")
        self.status_label.config(text=f"STOPPED.
{len(self.session_recorder)} data points recorded.",
foreground="red")
        self.start_stop_button.config(text="Start
PID")
        self.save_button.config(state=tk.NORMAL)
        self.is_running = False
    else:
        # Proses START
        if not
self.apply_pid_settings(is_starting=True):
            # Jangan mulai jika parameter awal tidak
valid

            return

        self._reset_session_data()
        self._send_command_to_unity("start_pid")
        self.status_label.config(text="Status: PID
Running", foreground="green")
        self.start_stop_button.config(text="Stop
PID")

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        self.save_button.config(state=tk.DISABLED)
        self.start_time = time.time()
        self.is_running = True

    def apply_pid_settings(self, is_starting=False):
        """Mengirim pengaturan PID ke Unity.
        Mengembalikan True jika sukses."""
        try:
            params = {
                'Kp': float(self.kp_var.get()),
                'Ki': float(self.ki_var.get()),
                'Kd': float(self.kd_var.get()),
                'Setpoint':
float(self.setpoint_var.get())
            }
        except ValueError:
            self.status_label.config(text="Error: Input
harus berupa angka valid.", foreground="red")
            messagebox.showerror("Input Error", "Pastikan
semua nilai Kp, Ki, Kd, dan Setpoint adalah angka.")
            return False

        if is_starting:
            self.session_pid_params = params

            self.current_setpoint = params['Setpoint']
            message =
f"pid,{params['Kp']},{params['Ki']},{params['Kd']},{param
s['Setpoint']}"
            self._send_command_to_unity(message)

        if not self.is_running:

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        self.status_label.config(text="PID settings
applied. Ready to start.", foreground="blue")
        return True

    def _send_command_to_unity(self, command):
        try:
            self.sock.sendto(command.encode('utf-8'),
(UNITY_IP, UNITY_PORT_SEND))
        except Exception as e:
            print(f"Gagal mengirim perintah '{command}':
{e}")
            self.status_label.config(text=f"Error sending
command: {e}", foreground="red")

    def _reset_session_data(self):
        self.plot_data.clear()
        self.session_recorder.clear()
        self._update_plot(None)
        print("Grafik dan data rekaman telah direset.")

    def _update_plot(self, frame):
        # 1. Proses antrian data
        if self.is_running:
            while not self.data_queue.empty():
                try:
                    timestamp, value =
self.data_queue.get_nowait()
                    self.plot_data.append(value)
                    elapsed_time = timestamp -
self.start_time
                    self.session_recorder.append({

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        "Waktu (s)": elapsed_time, "Sudut
Aktual": value, "Setpoint": self.current_setpoint
    })
    except queue.Empty:
        break

# 2. Gambar ulang plot
self.ax.clear()
self.ax.grid(True, linestyle='--', alpha=0.6)
self.ax.set_title("Sudut Real-time dari Unity")
self.ax.set_xlabel("Sampel (Gunakan toolbar di
bawah untuk scroll/pan/zoom)")
self.ax.set_ylabel("Sudut (derajat)")
self.ax.set_ylim(-10, 370)

if self.plot_data:
    self.ax.plot(range(len(self.plot_data)),
list(self.plot_data), 'r-', label='Sudut Aktual')
if self.is_running:
    self.ax.axhline(y=self.current_setpoint,
color='b', linestyle='--', label='Setpoint')

# Selalu tampilkan legenda jika ada label yang
didefinisikan
if self.ax.get_legend_handles_labels()[0]:
    self.ax.legend(loc='upper left')

# Perlu dipanggil untuk menggambar ulang canvas
self.canvas.draw_idle()

def save_to_excel(self):
    if not self.session_recorder:

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        messagebox.showwarning("No Data", "Tidak ada
data sesi untuk disimpan.")
        return

        filepath =
filedialog.asksaveasfilename(defaultextension=".xlsx",
filetypes=[("Excel Workbook", "*.xlsx")], title="Simpan
Sesi PID")
        if not filepath:
            self.status_label.config(text="Penyimpanan
dibatalkan.", foreground="orange")
            return

        self.status_label.config(text="Menyimpan ke
Excel...", foreground="blue")
        self.master.update_idletasks()

        try:
            df_data = pd.DataFrame(self.session_recorder)
            df_data['Waktu (s)'] = df_data['Waktu
(s)'].round(4)
            df_data['Sudut Aktual'] = df_data['Sudut
Aktual'].round(4)
            df_params =
pd.DataFrame([self.session_pid_params])

            with pd.ExcelWriter(filepath,
engine='openpyxl') as writer:
                df_data.to_excel(writer,
sheet_name='PID_Data', index=False)
                df_params.to_excel(writer,
sheet_name='PID_Parameters', index=False)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        self.status_label.config(text="Data berhasil
disimpan!", foreground="green")
        messagebox.showinfo("Sukses", f"Data berhasil
disimpan ke:\n{filepath}")
    except Exception as e:
        self.status_label.config(text=f"Error saat
menyimpan: {e}", foreground="red")
        messagebox.showerror("Save Error", f"Gagal
menyimpan file Excel.\n\nError: {e}")

def _on_closing(self):
    """Menangani penutupan jendela aplikasi dengan
bersih."""
    if messagebox.askokcancel("Keluar", "Apakah Anda
yakin ingin keluar?"):
        print("Menutup aplikasi...")
        self.stop_event.set()
        if self.sock:
            self.sock.close()
        # Beri waktu sedikit untuk thread selesai
        if hasattr(self, 'listener_thread'):
            self.listener_thread.join(timeout=1.5)
        self.master.destroy()

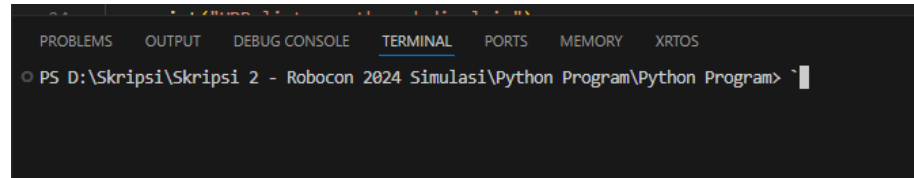
if __name__ == "__main__":
    try:
        root = tk.Tk()
        app = PIDControllerApp(root)
        root.mainloop()
    except Exception as e:
        print(f"Gagal menjalankan aplikasi: {e}")

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

12. Apabila terdapat paket yang belum diinstall, gunakan kombinasi keyboard shortcut (CTRL + `) untuk membuka terminal di VS Code.



13. Ketik sintaks di bawah ini, kemudian klik enter untuk setiap sintaks yang diketik untuk mengeksekusi instalasi *library* eksternal Python.

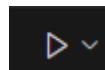
- a. pip install pandas

```
PS D:\Skripsi\Skripsi 2 - Robocon 2024 Simulasi\Python Program\Python Program> pip install pandas
```

- b. pip install matplotlib

```
PS D:\Skripsi\Skripsi 2 - Robocon 2024 Simulasi\Python Program\Python Program> pip install matplotlib
```

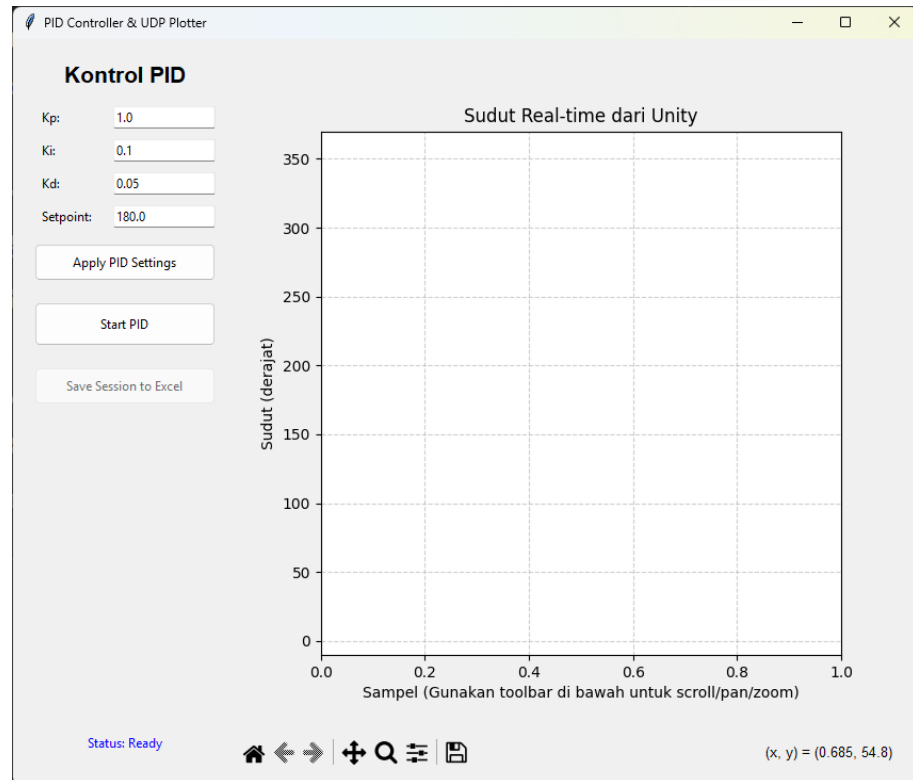
14. Setelah berhasil menginstall library eksternal di atas, selanjutnya jalankan program python dengan klik *button* Run Python File yang berada di sebelah kanan atas VS Code.



15. Tunggu hingga program *GUI* Python berjalan, maka akan tampak sebagai berikut.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit



16. Kembali ke program Unity dan jalankan dengan klik tombol seperti pada gambar di bawah ini.



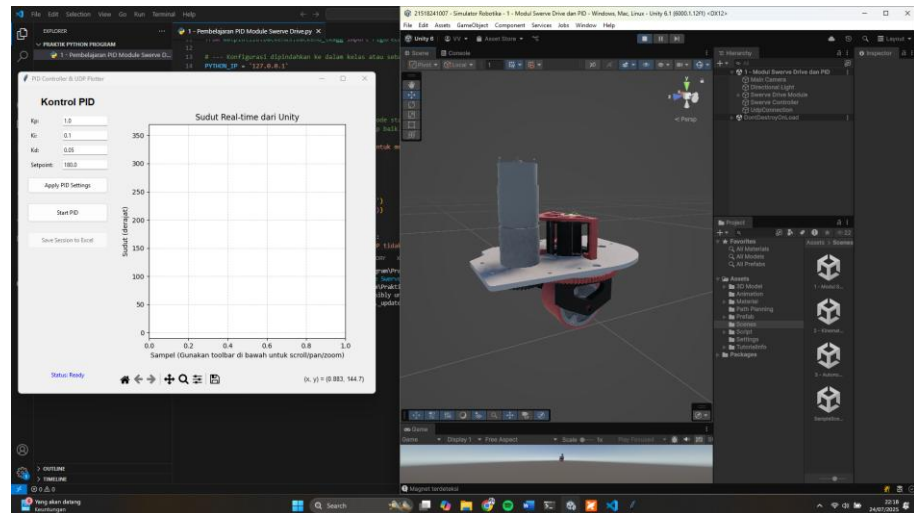
17. Setelah berhasil dijalankan, maka pada Unity tombol akan berganti seperti pada gambar berikut.



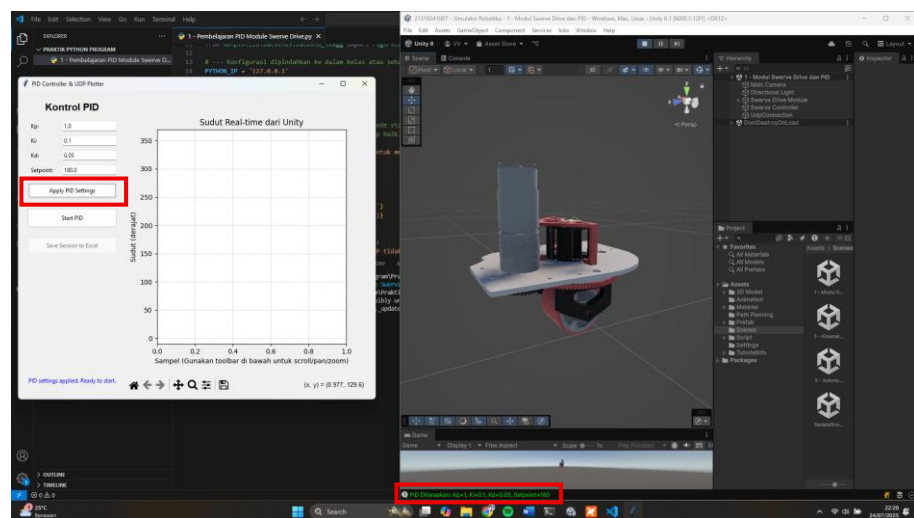
18. Gunakan *split screen* untuk mempermudah *tuning* parameter PID dan pengamatan simulasi *Swerve Drive*.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit



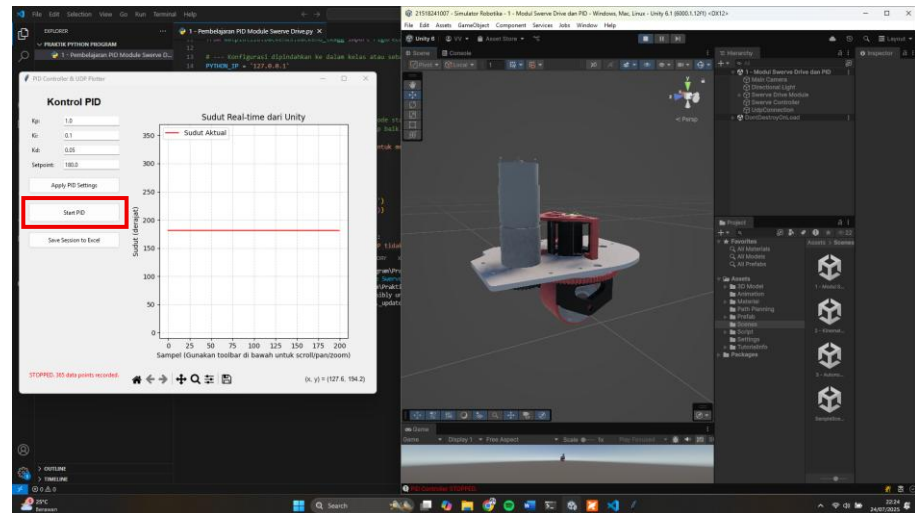
19. Klik pada “Apply PID Settings” untuk mengirim parameter PID ke Unity. Pastikan pada Unity terdapat *print* untuk PID yang telah diterapkan.



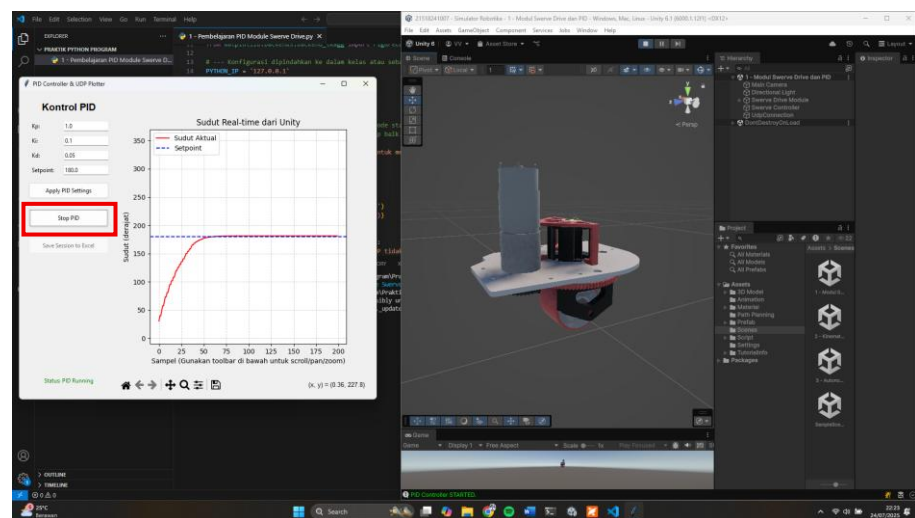
20. Setelah parameter berhasil dikirim ke Unity, langkah selanjutnya adalah memulai PID untuk menggerakkan *steering* dari *Swerve Drive*. Klik pada tombol “Start PID” untuk mengirim perintah memulai simulasi.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i> Menggunakan Metode PID	No. Job: 1
	Revisi: 00	Controller	4 x 50 menit



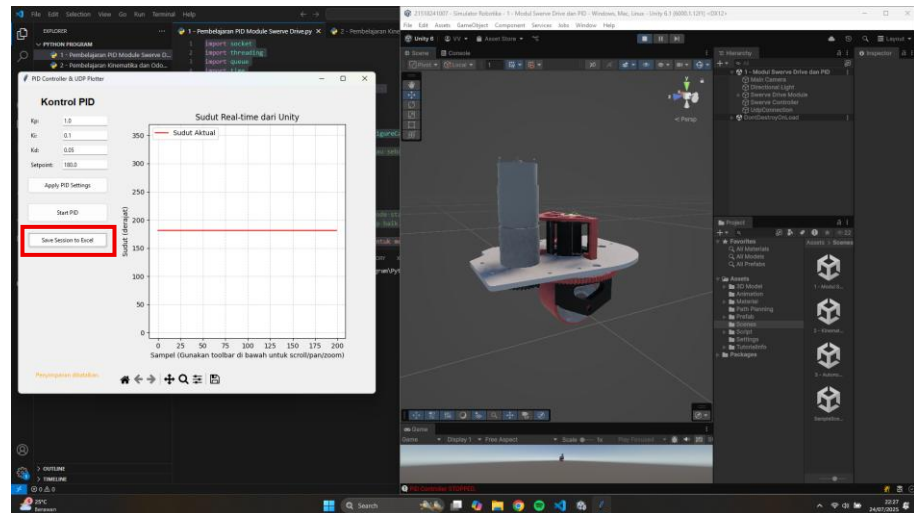
21. Setelah selesai melakukan simulasi klik pada “Stop PID” untuk menghentikan simulasi



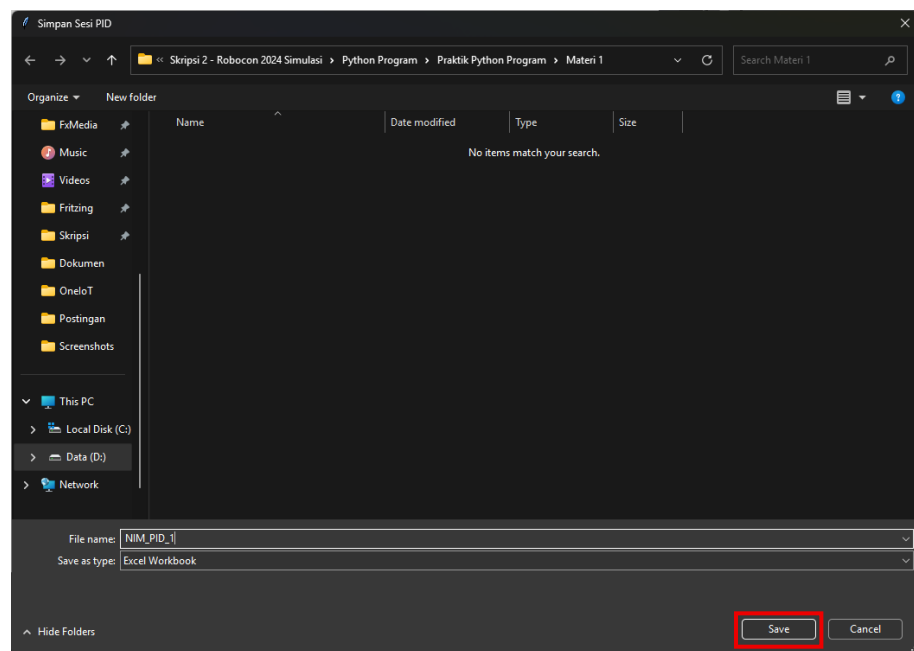
22. Jika ingin menyimpan hasil *monitoring data* sudut dari Swerve Drive, maka klik pada “Save Session to Excel”.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

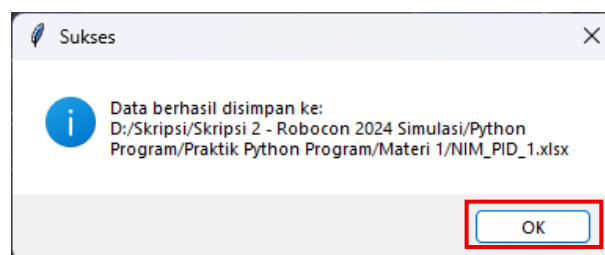
	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit



23. Setelah dialog *file explorer* muncul simpan dan beri nama filenya dengan NIM_PID_1, lalu simpan.



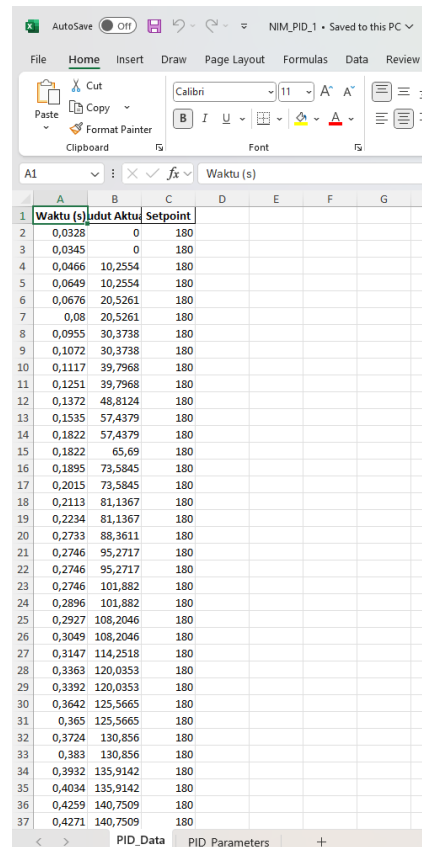
24. Apabila muncul dialog di bawah ini, tekan Ok untuk melanjutkan.



Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

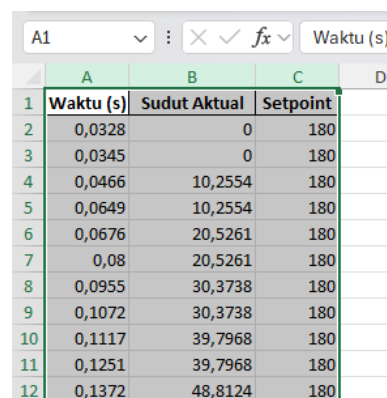
	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

25. Buka file excel yang sebelumnya telah disimpan. Terdapat dua buah *sheet* yang berisi masing-masing menyimpan hasil monitoring data PID dan parameter PID yang sebelumnya diaplikasikan ke *Swerve Drive*.



Waktu (s)	Sudut Aktual	Setpoint
0,0328	0	180
0,0345	0	180
0,0466	10,2554	180
0,0649	10,2554	180
0,0676	20,5261	180
0,08	20,5261	180
0,0955	30,3738	180
0,1072	30,3738	180
0,1117	39,7968	180
0,1251	39,7968	180
0,1372	48,8124	180
0,1535	57,4379	180
0,1822	57,4379	180
0,1822	65,69	180
0,1895	73,5845	180
0,2015	73,5845	180
0,2113	81,1367	180
0,2234	81,1367	180
0,2733	88,3611	180
0,2746	95,2717	180
0,2746	95,2717	180
0,2746	101,882	180
0,2896	101,882	180
0,2927	108,2046	180
0,3049	108,2046	180
0,3147	114,2518	180
0,3363	120,0353	180
0,3392	120,0353	180
0,3642	125,5665	180
0,365	125,5665	180
0,3724	130,856	180
0,383	130,856	180
0,3932	135,9142	180
0,4034	135,9142	180
0,4259	140,7509	180
0,4271	140,7509	180

26. Blok semua kolom A, B, dan C.

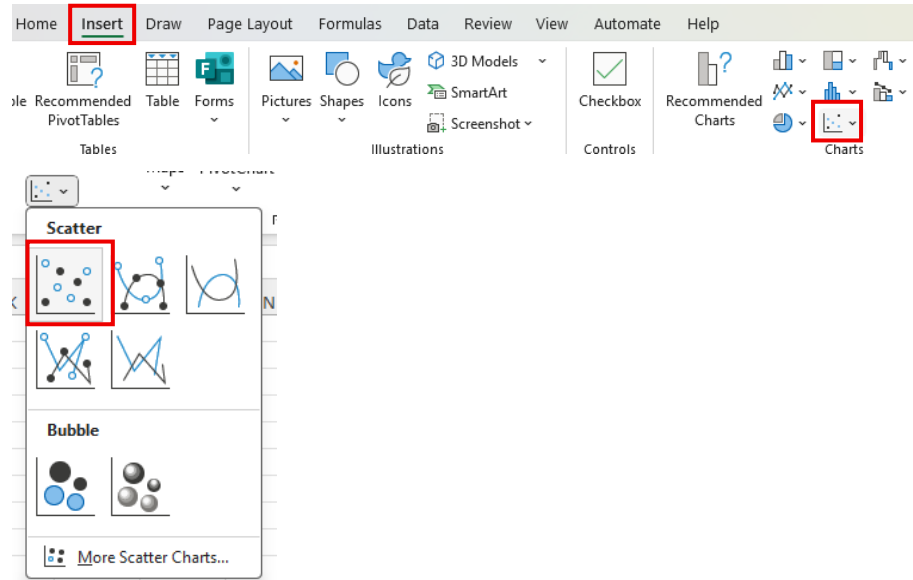


Waktu (s)	Sudut Aktual	Setpoint
0,0328	0	180
0,0345	0	180
0,0466	10,2554	180
0,0649	10,2554	180
0,0676	20,5261	180
0,08	20,5261	180
0,0955	30,3738	180
0,1072	30,3738	180
0,1117	39,7968	180
0,1251	39,7968	180
0,1372	48,8124	180

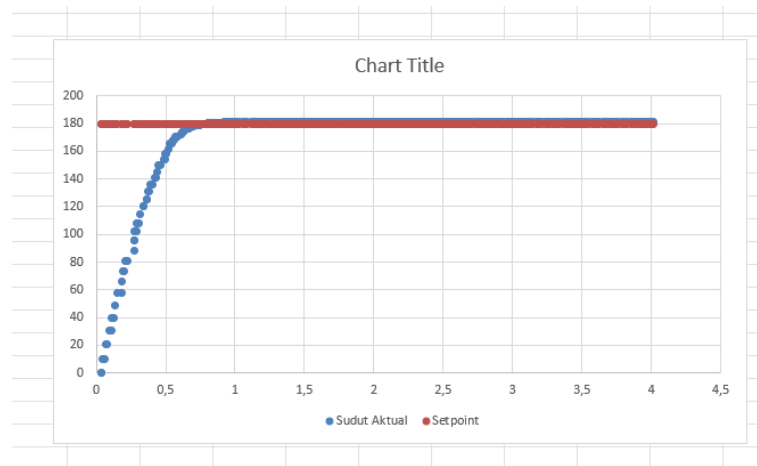
27. Buka tab Insert di excel, pada bagian *charts* pilih Scatter.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit



28. Grafik untuk respon sistem *Swerve Drive* akan terlihat seperti gambar di bawah ini.



29. Selanjutnya adalah menganalisis respon sistem *Rise Time*. Buatlah 5 parameter yaitu: *Steady State* (SS), 10% SS, 90% SS, 10% Time, serta 90% Time untuk menganalisis *Rise Time*.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

F
Steady State
10% SS
90% SS
10% Time
90% Time
Rise Time

30. Gunakan rumus di bawah ini untuk mencari nilai *Steady State*. Ambil sebanyak mungkin data dari Sudut Aktual yang nilainya stabil dan mendekati *setpoint*.

$$= \text{AVERAGE}(B_{xx} : B_{yy})$$

Di mana:

B_{xx} = Diganti dengan kolom di mana data pertama kali mencapai kestabilan,

B_{yy} = Diganti dengan kolom terakhir dari data sudut aktual.

31. Jika sintaks yang dimasukkan ke excel benar, maka akan menunjukkan nilai Steady State seperti pada gambar di bawah ini.

Steady State	181,25207
--------------	-----------

32. Selanjutnya untuk mencari nilai dari parameter 10% dan 90% dari *Steady State* bisa menggunakan rumus di bawah ini.

a. 10% SS

$$= 0,1 * (SS)$$

b. 90 % SS

$$= 0,9 * (SS)$$

Sebagai catatan, ganti nilai SS dengan kolom di mana nilai *Steady State* sebelumnya ditemukan. Misalkan pada kolom G baris 2 maka SS diganti menjadi G2. Terdapat perangkat yang menggunakan tanda koma (,) atau

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

tanda titik (.) sebagai nilai pecahan, sesuaikan hal ini agar tidak terjadi error pada sintaks.

10% SS	18,125207
90% SS	163,126863

33. Apabila telah ditemukan nilai 10% dan 90% dari *Steady State*, langkah berikutnya adalah menemukan nilai waktu mencapai 10% dan 90% dari *Steady State*. Gunakan sintaks di bawah ini untuk menemukan nilai parameter tersebut.

a. 10% Time

$$= \text{INDEX}(A2: A_t; \text{MATCH}(\text{TRUE}; B2: B_t \geq C_x; 0))$$

b. 90% Time

$$= \text{INDEX}(A2: A_t; \text{MATCH}(\text{TRUE}; B2: B_t \geq D_x; 0))$$

Terdapat parameter dicetak tebal yang perlu diganti, A_t diganti dengan nilai waktu terakhir apabila terdapat pada baris 366 maka diganti menjadi A366. Parameter B_t diganti dengan baris sudut aktual terakhir menjadi B366. Parameter C_x disesuaikan dengan letak kolom dan baris dari 10% SS yang sebelumnya telah ditemukan misalkan pada kolom G dan baris 3 maka menjadi G3, hal serupa dilakukan untuk parameter D_x namun parameter yang diperlukan adalah 90% SS.

10% Time	0,0676
90% Time	0,5299

34. Setelah menemukan semua parameter yang diperlukan, untuk mencari Rise Time, kurangi nilai dari 90% Time dengan 10% time.

$$\text{Rise Time} = 90\% \text{ Time} - 10\% \text{ Time}$$

$$\text{Rise Time} = 0,5299 - 0,0676$$

$$\text{Rise Time} = 0,4623 \text{ (s)}$$

Dengan mengurangi nilai tersebut, maka Rise Time ditemukan dengan nilai 0,4623 detik.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

35. Perhitungan selanjutnya adalah menemukan nilai *overshoot* dari sistem, cari terlebih dahulu nilai puncak dari sudut aktual. Gunakan sintaks berikut.

$$= MAX(B: B)$$

Sudut Peak	181,3453
-------------------	-----------------

36. Apabila sudut *overshoot* tertinggi telah ditemukan, selanjutnya gunakan sintaks berikut untuk menghitung nilai *overshoot* dari sistem dalam satuan persen (%).

$$= ((y_{\max} - \text{steady_state}) / \text{steady_state}) * 100$$

Ganti nilai $y_{\max}$ dengan sel Sudut Peak yang telah ditemukan sebelumnya dan nilai **steady_state** dengan baris dan kolom yang telah ditemukan sebelumnya. Maka ditemukan *overshoot* untuk sistem adalah sekitar 0,05%.

Overshoot	0,051436654
------------------	--------------------

37. Selanjutnya untuk mencari nilai *Settling Time* tentukan nilai toleransi yang akan digunakan untuk sistem misalkan menggunakan nilai $\pm 5\%$, maka tentukan batas toleransi atas dan bawah. Gunakan sintaks berikut untuk mencari nilai dari batas toleransi atas dan bawah tersebut.

a. Batas atas (+5%)

$$= SS \times (1 + 5\%)$$

b. Batas bawah (-5%)

$$= SS \times (1 - 5\%)$$

Ganti nilai SS dengan kolom dan baris di mana nilai *steady state* sebelumnya telah didapatkan.

5% +	190,3146735
5% -	172,1894665

38. Pada kolom D1 berilah judul kolom “Settle” yang menandakan apakah nilai sudut sudah mendekati rentang $\pm 5\%$ dari setpoint yang ditentukan.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

	A	B	C	D
1	Waktu (s)	Sudut Aktual	Setpoint	Settle

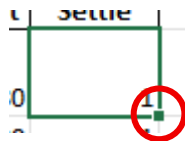
39. Setelah memberi judul kolom, gunakan sintaks berikut untuk menentukan apakah nilai sudut aktual sudah berada di dalam nilai batas atas dan bawah di sel D2.

$$= \text{IF}(\text{AND}(\text{B2} \geq \text{A}_x; \text{B2} \leq \text{B}_x * (1 + 5\%)); 0; 1)$$

Ganti A_x dengan nilai di mana batas bawah -5% ditemukan misal berada di sel G15 maka ganti dengan \$G%15, hal yang sama untuk parameter B_x ganti dengan sel yang memuat batas atas +5%.

	A	B	C	D
1	Waktu (s)	Sudut Aktual	Setpoint	Settle
2	0,0328	0	180	1

40. Klik dua kali pada titik yang dilingkari pada gambar di bawah ini.



41. Maka semua nilai otomatis terisi oleh nilai 0 atau 1, apabila bernilai 0 dapat dikatakan bahwa nilai tersebut sudah berada dalam batas yang ditentukan, sebaliknya jika bernilai 1 maka belum berada dalam batas.

	A	B	C	D
1	Waktu (s)	Sudut Aktual	Setpoint	Settle
2	0,0328	0	180	1
3	0,0345	0	180	1
4	0,0466	10,2554	180	1
5	0,0649	10,2554	180	1
6	0,0676	20,5261	180	1
7	0,08	20,5261	180	1
8	0,0955	30,3738	180	1
9	0,1072	30,3738	180	1
10	0,1117	39,7968	180	1

42. Carilah nilai transisi antara 1 menjadi 0 untuk menentukan nilai *settling time*.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

	A	B	C	D
48	0,5592	168,2976	180	1
49	0,559	168,2976	180	1
50	0,5702	170,6771	180	1
51	0,5872	170,6771	180	1
52	0,6126	172,6251	180	0

43. Setelah ditemukan nilai transisi dari 1 ke 0 pertama, maka ditemukan nilai *settling time* yaitu 0,6126 detik.

44. Penentuan nilai *Steady State Error* (SSE) bisa menggunakan rumus di bawah ini:

$$SSE = Setpoint - Steady_State$$

$$SSE = 180 - 181,25207$$

$$SSE = -1,25207$$

45. Apabila SSE telah ditemukan, maka respon sistem dapat diketahui. SSE positif menunjukkan bahwa sistem belum mencapai target. SSE negatif menunjukkan bahwa sistem melewati target yang ditentukan. Apabila nilai SSE 0 maka sistem dapat dikatakan mencapai setpoint ideal.

G. Bahan Diskusi

1. Berikut di bawah ini merupakan kode dari program untuk mengendalikan *Swerve Drive* menggunakan PID, analisislah bagaimana cara program bekerja dan *flow* pemrograman untuk mengendalikan putaran motor *steering* pada *Swerve Drive*.

a. Swerve Drive Script

```
using UnityEngine;
using UnityEngine.Serialization;

namespace Script
{
    public class SwerveDrive : MonoBehaviour
    {
        [Header("Components")] public AS5600 as5600;
        public PG45 pg45;
        public GameObject pivotRotasi;
    }
}
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

[FormerlySerializedAs("rodaSwerve")] public
BLDC swerveWheel;

[Range(0, 360f)] public float setAngle { get;
private set; }

[Header("Swerve Parameter")] public float
angleOffset;

public int rasioGearPG45;
public int rasioGearSwerveDrive;
public bool useNearestRotation;

[FormerlySerializedAs("kP")] [Header("PID
Parameters")] [Range(0, 10f)]
public float proportionalGain;

[FormerlySerializedAs("kI")] [Range(0, 10f)]
public float integralGain;

[FormerlySerializedAs("kD")] [Range(0, 10f)]
public float derrivativeGain;

[Header("Motor Parameters")] [Range(0, 600)]
public float maximumSwerveBLDCSpeed;

[Range(0, 1200)] public float
maximumSteeringSpeed;

public const float WheelDiameter = 0.07f; // 70
mm

private int _wheelDirection = 1;

private PID _pid = new PID();

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

void Update()
{
    swerveWheel.SetMaxSpeed(maximumSwerveBLDCSpeed);
    pg45.SetMaxSpeed(maximumSteeringSpeed);
    _pid.SetParameters(proportionalGain,
    integralGain, derrivativeGain, maximumSteeringSpeed);
}

private void FixedUpdate()
{
    UpdateAngle();
}

private float _pg45Incremental;

private void UpdateAngle()
{
    float rasioGear =
(float)rasioGearSwerveDrive / rasioGearPG45;

    float currentAngle =
Mathf.Repeat(as5600.ReadAngle() + angleOffset, 360f);
    float error;

    if (useNearestRotation)
    {
        error = Mathf.DeltaAngle(currentAngle,
setAngle); // Standard nearest rotation handling
    }
    else
    {
        // Manual wraparound correction
        error = setAngle - currentAngle;
    }
}

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

        if (error > 180f)
            error -= 360f; // Correct
wraparound from 359 → 0
        else if (error < -180f)
            error += 360f; // Correct
wraparound from 0 → 359
        }

        float outputPid = _pid.GetOutput(error,
Time.fixedDeltaTime);
        DriveMotorSteering(outputPid);

        pivotRotasi.transform.rotation =
            Quaternion.Euler(0,
Mathf.Repeat(pg45.GetIncrementalRotation() / rasioGear,
360f), 0);

        // Debug.Log(
        //     $"now : {currentAngle}, raw set :
{_rawSetAngle}, set : {setAngle}, incremental :
{pg45.GetIncrementalRotation()}, error {error}, output
{outputPid}");
    }

    private float _rawSetAngle;

    public void SetAngle(float angleSet)
    {
        if (!useNearestRotation)
        {
            _wheelDirection = 1;
            setAngle = angleSet;
            return;
        }
    }

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

float currentAngle = as5600.ReadAngle();

float setpoint = angleSet;
float setpointFlip = Mathf.Repeat(angleSet
+ 180f, 360f);

float deltaToSetpoint =
Mathf.DeltaAngle(currentAngle, setpoint);
float deltaToSetpointFlip =
Mathf.DeltaAngle(currentAngle, setpointFlip);

if (Mathf.Abs(deltaToSetpoint) <
Mathf.Abs(deltaToSetpointFlip))
{
    setAngle = setpoint;
    _wheelDirection = 1;
}
else
{
    setAngle = setpointFlip;
    _wheelDirection = -1;
}

_rawSetAngle = angleSet;
}

public float GetAngle() => as5600.ReadAngle() +
angleOffset;

public float GetWheelSpeedRpm() =>
swerveWheel.GetCurrentSpeed();

public float CalculateWheelSpeed()
{
    float circumference = Mathf.PI *

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

WheelDiameter; // Correct circumference formula

float rps = swerveWheel.GetCurrentSpeed() /
60f; // Convert RPM to RPS
return rps * circumference;
}

public void UseNearestRotation(bool use) =>
useNearestRotation = use;

public void SetMaximumSteeringSpeed(float
maxSteeringSpeed) => maximumSteeringSpeed =
maxSteeringSpeed;

// ReSharper disable once InconsistentNaming
public void SetMaximumBLDCSpeed(float
maxBldcSpeed) => maximumSwerveBLDCSpeed = maxBldcSpeed;
private void DriveMotorSteering(float output)
=> pg45.SetSpeed(output);

public void SetSpeed(float speed) =>
swerveWheel.SetSpeed(speed * _wheelDirection);

public void SetPidParameters(PID pid)
{
    proportionalGain = pid.Kp;
    integralGain = pid.Ki;
    derrivativeGain = pid.Kd;
}
}

```

b. Swerve Controller Script

```

using System;
using System.Globalization;
using UnityEngine;
using UnityEngine.Serialization;

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```
namespace Script
{
    public class SwerveController : MonoBehaviour
    {
        public SwerveDrive swerveDrive;
        public UdpConnection udpConnection;

        [Header("Input")] public bool useKeyboard;
        [Range(-1, 1)] public float forwardInput;
        [Range(-1, 1)] public float strafeInput;
        [Range(-1, 1)] public float rotateInput;

        [Header("Swerve Drive")] public bool
useNearestRotation;

        public bool testSwerve;
        [Range(0, 360f)] public float setSwerveAngle;
        [Range(0, 600f)] public float setSwerveSpeed;

        [FormerlySerializedAs("maximumSwerveBLDCSpeed")]
        [Header("Motor Parameters")] [Range(0, 600)]
        public float maximumWheelSpeed;

        [Range(0, 1200)] public float
maximumSteeringSpeed;

        private float _angleNow = 0.0f;
        private float _lastSetAngle = 0.0f;

        private void Update()
        {
            float forward;
            float strafe;
            float rotation;
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

swerveDrive.SetMaximumSteeringSpeed(maximumSteeringSpeed);

swerveDrive.SetMaximumBLDCSpeed(maximumWheelSpeed);

swerveDrive.UseNearestRotation(useNearestRotation);

    if (!testSwerve)
    {
        if (useKeyboard)
        {
            forward = Input.GetKey(KeyCode.W) ?
1.0f : Input.GetKey(KeyCode.S) ? -1.0f : 0f;
            strafe = Input.GetKey(KeyCode.A) ?
-1.0f : Input.GetKey(KeyCode.D) ? 1.0f : 0f;
            rotation = Input.GetKey(KeyCode.Q)
? 1.0f : Input.GetKey(KeyCode.E) ? -1.0f : 0f;
        }
        else
        {
            forward = forwardInput;
            strafe = strafeInput;
            rotation = rotateInput;
        }

        double xInput = strafe;
        double yInput = forward;
        double thetaInput = -rotation;

        double adjustVx = xInput *
Math.Cos(_angleNow) + yInput * Mathf.Sin(_angleNow);
        double adjustVy = -xInput *
Math.Sin(_angleNow) + yInput * Math.Cos(_angleNow);

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

double xSpeed = adjustVx - thetaInput;
double ySpeed = adjustVy + thetaInput;

float calculatedAngle =
Mathf.Repeat((float)(Mathf.Rad2Deg *
(Mathf.Atan2(ySpeed, xSpeed))), 360f);
float calculatedSpeed =
(float)Math.Sqrt(Math.Pow(xSpeed, 2) + Math.Pow(ySpeed,
2));

if
(!Mathf.Approximately(calculatedAngle, _lastSetAngle))
{
swerveDrive.SetAngle(calculatedAngle);
}
else
{
swerveDrive.SetAngle(_lastSetAngle);
}

swerveDrive.SetSpeed(calculatedSpeed *
maximumWheelSpeed);
_lastSetAngle = calculatedAngle;
}
else if(testSwerve &&
udpConnection.IsPidRunning)
{
swerveDrive.SetAngle(setSwerveAngle);
swerveDrive.SetSpeed(setSwerveSpeed);

udpConnection.SendTelemetry(swerveDrive.GetAngle().ToSt

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

```

ring(CultureInfo.InvariantCulture));
    }

    }

    public void SetSetpoint(float setpoint)
    {
        setSwerveAngle = setpoint;
    }
}

```

Apabila diperlukan, gunakan *file* program lain. Untuk membuka *Script* lain, gunakan Unity menuju ke *Assets > Script*.

- Isilah tabel di bawah ini dengan mensimulasikan kembali perputaran roda *steering* menggunakan parameter PID yang berbeda. Kemudian setelah selesai mengamati hasil simulasi dan kondisi roda, analisislah parameter respon sistem yang terjadi untuk parameter PID tersebut.

Tabel 1. Respon Sistem Kp 1, Ki 0.1, Kd 0.05

Nilai Kp = 1 Ki = 0,1 Kd = 0,05					
Sudut Awal	Sudut Target	Rise Time	Over-shoot	Settling Time	Steady State Error
0	180	0,46 s	0,05	0,61 s	-1,25

Kondisi respon sistem:

Underdamped, sistem naik cukup cepat, namun terjadi sedikit overshoot sebelum nilai akhirnya mendekati setpoint

Tabel 2. Respon Sistem Praktikum 1

Nilai Kp =, Ki =, Kd =					
Sudut Awal	Sudut Target	Rise Time	Over-shoot	Settling Time	Steady State Error

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

--	--	--	--	--	--

Kondisi respon sistem:

.....

.....

.....

Tabel 3. Respon Sistem Praktikum 2

Nilai Kp =, Ki =, Kd =					
Sudut Awal	Sudut Target	<i>Rise Time</i>	<i>Over-shoot</i>	<i>Settling Time</i>	<i>Steady State Error</i>

Kondisi respon sistem:

.....

.....

.....

Tabel 4. Respon Sistem Praktikum 3

Nilai Kp =, Ki =, Kd =					
Sudut Awal	Sudut Target	<i>Rise Time</i>	<i>Over-shoot</i>	<i>Settling Time</i>	<i>Steady State Error</i>

Kondisi respon sistem:

.....

.....

.....

Tabel 5. Respon Sistem Praktikum 2

Nilai Kp =, Ki =, Kd =
--

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 1	LS 1: Simulasi Pengendalian Modul <i>Swerve Drive</i>	No. Job: 1
	Revisi: 00	Menggunakan Metode PID Controller	4 x 50 menit

Sudut Awal	Sudut Target	<i>Rise Time</i>	<i>Over-shoot</i>	<i>Settling Time</i>	<i>Steady State Error</i>

Kondisi respon sistem:

.....

.....

.....

- Jelaskan bagaimana hubungan K_p , K_i , dan K_d terhadap karakteristik respon sistem (*rise time*, *overshoot*, *settling time*, *steady-state error*) berdasarkan data pada tabel di atas.
- Buatlah rangkuman mengenai praktikum yang telah dilakukan, dimulai dengan penjelasan bagaimana pengendalian steering *Swerve Drive* hingga respon sistem.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

A. Kompetensi

Mahasiswa mampu memahami dan mengaplikasikan persamaan kinematika *Swerve Drive Mobile Robot* empat roda dan konsep odometri untuk menghitung posisi serta orientasi robot.

B. Sub Kompetensi

Setelah melaksanakan praktikum, mahasiswa diharapkan mampu:

1. Menjelaskan prinsip kerja kinematika pada *Swerve Drive* robot empat roda.
2. Menerapkan persamaan kinematika pada *Swerve Drive* robot empat roda dalam pemrograman.
3. Memahami konsep dasar odometri untuk menentukan posisi dan orientasi robot.
4. Mengimplementasikan simulasi kinematika dan odometri pada Unity 3D.
5. Menganalisis hasil simulasi untuk mengevaluasi pergerakan dan estimasi posisi robot.

C. Dasar Teori

1. Model Kinematika Swerve Drive

Kinematika adalah studi tentang gerak tanpa mempertimbangkan penyebab gerak termasuk gaya dan torsi. Dalam konteks *Swerve Drive*, kinematika menghubungkan kecepatan dan sudut setiap roda dengan kecepatan gerak robot secara keseluruhan. Terdapat dua jenis kinematika antara lain:

- a. Kinematika Balik (*Inverse Kinematics*) adalah kinematika yang menentukan kecepatan dan sudut setiap roda individu (v_i , θ_i) berdasarkan *input* gerak robot yang diinginkan (v_x , v_y , ω). Perhitungan kinematika ini digunakan untuk mengendalikan robot dan menjadi inti dari logika gerak robot.
- b. Kinematika Maju (*Forward Kinematics*) adalah kinematika yang menentukan gerak robot secara keseluruhan (v_x , v_y , ω) berdasarkan

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

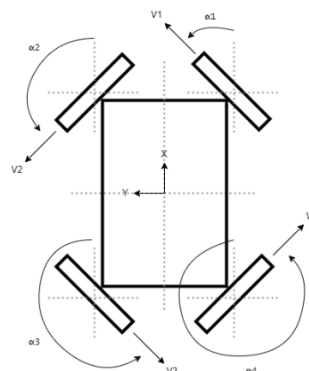
data kecepatan dan sudut dari setiap roda individu (v_i , θ_i).

Kinematika ini berguna untuk estimasi posisi (odometri).

Swerve Drive memungkinkan setiap roda untuk berotasi terhadap sumbu vertikalnya untuk berbelok dan secara bersamaan berputar pada sumbu horizontalnya untuk bergerak. Kombinasi dua gerakan ini menghasilkan mobilitas *omnidirectional* untuk mengendalikan gerakan secara presisi dengan pemodelan matematika.

Gambar 6. Kinematika Swerve Drive 4 Roda

(Sumber: <https://www.petrikvandervelde.nl/posts/Swerve-drive-kinematics-simulation>)



Sebelum menurunkan persamaan kinematika, penting untuk mendefinisikan kerangka acuan dan notasi yang akan digunakan, modul swerve drive disusun dalam konfigurasi persegi panjang seperti ilustrasi gambar di atas. Notasi yang digunakan antara lain:

- Kerangka acuan robot (*Body Frame*), merupakan titik pada koordinat (0,0) berada di tengah geometris dari sasis.
- Sumbu koordinat menunjukkan arah gerak robot, sumbu +X menunjukkan ke arah kanan robot, sedangkan sumbu +Y menunjukkan arah ke depan robot. Rotasi diukur mengelilingi sumbu Z yang menunjuk ke atas.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive</i>	No. Job: 2
	Revisi: 00	<i>Mobile Robot</i>	4 x 50 menit

- c. Terdapat dua parameter sasis antara lain L yang merupakan jarak total antara roda depan dan belakang (panjang) dan W yang mendefinisikan jarak total antara roda kiri dan kanan (lebar).
- d. Vektor kecepatan robot adalah kecepatan gerak robot yang diwakili dengan tiga komponen antara lain v_x yang merupakan kecepatan translasi ke arah samping sepanjang sumbu X, v_y yang mewakili kecepatan translasi ke arah depan sepanjang sumbu Y, dan ω merupakan kecepatan rotasi (angular) mengelilingi pusat robot.

Setiap modul roda dapat diidentifikasi berdasarkan posisinya Depan Kanan (FR), Depan Kiri (FL), Belakang Kiri (RL), dan Belakang Kanan (RR). *Inverse kinematics* bertujuan untuk menghitung kecepatan (v_i) dan sudut (θ_i) yang harus dicapai oleh setiap modul roda untuk menghasilkan gerakan robot yang diinginkan. Gerakan pada setiap titik benda tegar merupakan gabungan dari gerak translasi dan rotasi. Oleh karena itu, vektor kecepatan pada lokasi setiap modul roda (v_i) adalah jumlah vektor kecepatan translasi seluruh robot dan vektor kecepatan tangensial yang disebabkan oleh rotasi robot, dapat dirumuskan menjadi

$$v_i = v_{translasi} + v_{rotasi}$$

Keterangan:

v_i = Vektor kecepatan tiap roda

Vektor kecepatan rotasi dihitung sebagai produk silang antara kecepatan angular dan vektor posisi (jarak dari pusat robot ke roda). Dalam sistem 2 dimensi, hal ini dapat disederhanakan lebih lanjut menjadi:

$$v_{ix} = v_x - \omega * r_{iy}$$

$$v_{iy} = v_y + \omega * r_{ix}$$

Keterangan:

v_{ix} = Kecepatan roda ke-i pada sumbu X

v_{iy} = Kecepatan roda ke-i pada sumbu Y

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

v_x = Kecepatan translasi pusat robot pada sumbu X

v_y = Kecepatan translasi pusat robot pada sumbu Y

r_{ix} = Jarak roda ke-i terhadap pusat robot pada sumbu X

r_{iy} = Jarak roda ke-i terhadap pusat robot pada sumbu Y

Dalam hal ini, r_{ix} dan r_{iy} adalah koordinat posisi X dan Y dari roda ke-i relatif terhadap pusat robot. Berdasarkan parameter sasis L dan W, posisi setiap roda adalah:

$$\text{Depan Kanan (FR)} = \left(\frac{L}{2}, \frac{W}{2} \right)$$

$$\text{Depan Kiri (FL)} = \left(-\frac{L}{2}, \frac{W}{2} \right)$$

$$\text{Belakang Kiri (RL)} = \left(-\frac{L}{2}, -\frac{W}{2} \right)$$

$$\text{Belakang Kanan (RR)} = \left(\frac{L}{2}, -\frac{W}{2} \right)$$

Keterangan:

L = Panjang sasis robot (jarak antara roda depan dan belakang)

W = Lebar sasis robot (jarak antara roda kanan dan kiri)

Dengan mensubstitusikan koordinat-koordinat ini ke dalam persamaan umum, dapat ditentukan vektor kecepatan untuk setiap roda, antara lain:

a. Persamaan roda depan kanan (FR)

$$v_{FRx} = v_x - \omega * \left(\frac{W}{2} \right)$$

$$v_{FRy} = v_y + \omega * \left(\frac{L}{2} \right)$$

$$v_{FR} = \sqrt{\left(v_x - \omega * \left(\frac{W}{2} \right) \right)^2 + \left(v_y + \omega * \left(\frac{L}{2} \right) \right)^2}$$

$$\theta_{FR} = \arctan2 \left(\frac{v_y + \omega \cdot \frac{L}{2}}{v_x - \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

v_{FRx} = Kecepatan roda depan kanan pada sumbu x

v_{FRy} = Kecepatan roda depan kanan pada sumbu y

v_{FR} = Resultan kecepatan roda kanan, gabungan dari v_{FRx} dan v_{FRy}

θ_{FR} = Sudut kemudi roda depan kanan terhadap sumbu x

b. Persamaan roda depan kiri (FL)

$$v_{FLx} = v_x - \omega * \left(\frac{W}{2}\right)$$

$$v_{FLy} = v_y - \omega * \left(\frac{L}{2}\right)$$

$$v_{FL} = \sqrt{\left(v_x - \omega * \left(\frac{W}{2}\right)\right)^2 + \left(v_y - \omega * \left(\frac{L}{2}\right)\right)^2}$$

$$\theta_{FL} = \arctan2 \left(\frac{v_y - \omega \cdot \frac{L}{2}}{v_x - \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

v_{FLx} = Kecepatan roda depan kiri pada sumbu x

v_{FLy} = Kecepatan roda depan kiri pada sumbu y

v_{FL} = Resultan kecepatan roda kiri, gabungan dari v_{FLx} dan v_{FLy}

θ_{FL} = Sudut kemudi roda depan kiri terhadap sumbu x

c. Persamaan roda belakang kiri (RL)

$$v_{RLx} = v_x + \omega * \left(\frac{W}{2}\right)$$

$$v_{RLy} = v_y - \omega * \left(\frac{L}{2}\right)$$

$$v_{RL} = \sqrt{\left(v_x + \omega * \left(\frac{W}{2}\right)\right)^2 + \left(v_y - \omega * \left(\frac{L}{2}\right)\right)^2}$$

$$\theta_{RL} = \arctan2 \left(\frac{v_y - \omega \cdot \frac{L}{2}}{v_x + \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

v_{RLx} = Kecepatan roda belakang kiri pada sumbu x

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

v_{RLy} = Kecepatan roda belakang kiri pada sumbu y

v_{RL} = Resultan kecepatan roda belakang kiri, gabungan dari v_{RLx} dan v_{RLy}

θ_{RL} = Sudut kemudi roda belakang kiri terhadap sumbu x

d. Persamaan roda belakang kanan (RR)

$$v_{RRx} = v_x + \omega * \left(\frac{W}{2}\right)$$

$$v_{RRy} = v_y + \omega * \left(\frac{L}{2}\right)$$

$$v_{RR} = \sqrt{\left(v_x + \omega * \left(\frac{W}{2}\right)\right)^2 + \left(v_y + \omega * \left(\frac{L}{2}\right)\right)^2}$$

$$\theta_{RR} = \arctan2 \left(\frac{v_y + \omega \cdot \frac{L}{2}}{v_x + \omega \cdot \frac{W}{2}} \right)$$

Keterangan:

v_{RLx} = Kecepatan roda belakang kiri pada sumbu x

v_{RLy} = Kecepatan roda belakang kiri pada sumbu y

v_{RL} = Resultan kecepatan roda belakang kiri, gabungan dari v_{RLx} dan v_{RLy}

θ_{RL} = Sudut kemudi roda belakang kiri terhadap sumbu x

Dengan demikian, kecepatan dan sudut setiap roda dapat ditulis secara lebih ringkas:

a. Roda depan kanan (FR)

$$V_{FR} = \sqrt{(B^2 + D^2)}$$

$$\theta_{FR} = \text{atan2} \left(\frac{D}{B} \right)$$

b. Roda depan kiri (FL)

$$V_{FL} = \sqrt{(A^2 + D^2)}$$

$$\theta_{FL} = \text{atan2} \left(\frac{D}{A} \right)$$

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

c. Roda belakang kiri (RL)

$$V_{RL} = \sqrt{(A^2 + C^2)}$$

$$\theta_{RL} = \text{atan2} \left(\frac{C}{A} \right)$$

d. Roda belakang kanan (RR)

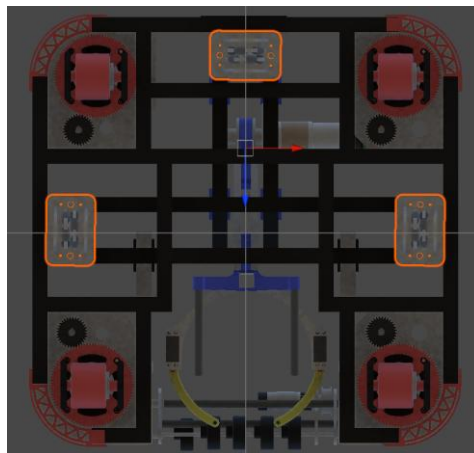
$$V_{RR} = \sqrt{(B^2 + C^2)}$$

$$\theta_{RR} = \text{atan2} \left(\frac{C}{B} \right)$$

Pemodelan yang disederhanakan di atas memungkinkan secara matematis dan lebih efisien untuk dihitung dalam perangkat lunak simulator.

2. Lokalisasi Odometry Tiga Roda

Gambar 7. Konfigurasi Odometry Tiga Roda



Odometry merupakan metode yang digunakan dalam memperkirakan posisi dan orientasi robot pada lapangan berdasarkan sensor seperti *wheel encoder*. Teknik ini digunakan untuk navigasi robotika, terutama untuk *mobile robot* yang bergerak di lingkungan yang diketahui maupun belum diketahui sebelumnya (Fachrizi et al., 2024). Odometry digunakan untuk memperkirakan posisi relatif terhadap posisi awal, digunakan sensor *rotary encoder* yang menghasilkan perhitungan jumlah pulsa yang dihasilkan kemudian dikonversi menjadi satuan

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

milimeter. Berikut merupakan persamaan yang digunakan untuk menentukan:

$$\text{Keliling roda } (K) = 2 * \pi * r$$

$$\text{Pulsa per mm} = \frac{\text{Resolusi Encoder}}{K}$$

Robot yang menggunakan roda *omnidirectional* dapat digunakan persamaan untuk mendapatkan koordinat X dan Y. Pada sistem omni tiga roda, perhitungan posisi menggunakan model kinematika *omnidirectional*, di mana kecepatan tiap roda diolah untuk mendapatkan perubahan posisi ($\Delta x, \Delta y$) dan perubahan orientasi ($\Delta \theta$). Akurasi odometry dapat ditingkatkan dengan menggabungkan nilai orientasi dari sensor lain seperti IMU, sehingga dapat meminimalisasi error akibat slip roda atau permukaan yang tidak rata. Selain itu, algoritma seperti *Extended Kalman Filter (EKF)* juga dapat digunakan untuk mengurangi akumulasi error yang umum terjadi pada odometry berbasis roda saja. Hal yang mempengaruhi error biasanya terjadi karena permukaan yang licik atau tidak rata, sehingga terjadi selip pada roda *encoder*.

Setiap pulsa pada *encoder* mewakili jarak tempuh tertentu, parameter lain yang menentukan perhitungan adalah keliling roda dan jumlah pulsa per putaran (PPR) pada *encoder*. Sehingga, untuk menentukan jarak yang ditempuh oleh roda dapat menggunakan persamaan:

$$\text{Jarak} = \text{Jumlah Pulsa} * \left(\frac{\text{Keliling Roda}}{\text{PPR}} \right)$$

Keterangan:

PPR = Pulsa per rotasi pada encoder

Parameter yang diperoleh menggunakan kinematika odometry tiga roda adalah: posisi robot (x, y, θ) mengacu pada kerangka robot di lingkungan yang tetap. Salah satu konfigurasi yang umum digunakan adalah tiga roda *encoder* pasif, di mana sistem ini terdiri dari tiga roda *tracking* yang tidak digerakkan oleh motor, namun berfungsi sebagai

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive</i>	No. Job: 2
	Revisi: 00	<i>Mobile Robot</i>	4 x 50 menit

sensor gerak translasi dan rotasi. Roda-roda tersebut ditempatkan dengan konfigurasi dua roda tegak lurus satu sama lain untuk mengukur pergerakan pada sumbu X dan Y, serta satu roda yang digunakan untuk mendeteksi perubahan orientasi pada robot. Dengan pendekatan kinematika planar, odometry tiga roda memungkinkan estimasi posisi dalam bidang dua dimensi.

Perubahan posisi dan orientasi dihitung berdasarkan kombinasi dari perubahan posisi roda-roda tersebut. Orientasi robot (θ) dapat ditentukan dari selisih gerakan roda-roda yang sejajar dengan sumbu utama, dikaitkan dengan lebar antar roda atau *offset* posisi roda terhadap pusat rotasi robot:

$$\Delta\theta = \frac{\Delta s_{kanan} - \Delta s_{kiri}}{\text{jarak antar roda}}$$

Keterangan:

$\Delta\theta$ = Perubahan sudut orientasi robot

Δs = Perubahan jarak tempuh roda

Perubahan posisi lateral (Δy) diukur oleh roda ketiga, yang posisinya tegak lurus terhadap dua roda sebelumnya. Karena robot juga mengalami gerakan rotasi, nilai perubahan perlu dilakukan koreksi dengan mengurangi komponen gerakan akibat rotasi, sehingga menjadi:

$$\Delta y = \Delta s_{tengah} - (\Delta\theta * d)$$

Keterangan:

Δy = Perubahan posisi lateral sumbu y

$\Delta\theta$ = Perubahan sudut orientasi robot

Δs_{tengah} = Jarak tempuh roda tengah

d = Jarak roda tengah terhadap pusat rotasi robot

Dengan d adalah jarak roda pengukur tengah terhadap pusat rotasi robot. Setelah komponen Δx , Δy dan $\Delta\theta$ dihitung, posisi robot dalam

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive</i>	No. Job: 2
	Revisi: 00	<i>Mobile Robot</i>	4 x 50 menit

koordinat global dapat diketahui dengan menggunakan transformasi berdasarkan orientasi saat itu:

$$x_{baru} = x_{lama} + \Delta x \cdot \cos(\theta) - \Delta y \cdot \sin(\theta)$$

$$y_{baru} = y_{lama} + \Delta x \cdot \sin(\theta) + \Delta y \cdot \cos(\theta)$$

Model ini bekerja dengan mengasumsikan bahwa permukaan lintasan robot rata dan tidak terjadi selip pada roda. Pada kenyataannya, akurasi odometry dapat terpengaruh oleh berbagai faktor seperti selip pada roda, ketidakrataan permukaan, atau akumulasi kesalahan dari integrasi data. Sehingga untuk mengurangi error yang terjadi, sistem odometry sering dikombinasikan dengan sensor tambahan seperti IMU dan dilengkapi dengan algoritma filter seperti *Kalman Filter* untuk mengurangi error secara progresif.

D. Alat dan Bahan

1. Laptop dan komputer
2. Unity 3D
3. *Software* Visual Studio Code

E. Keselamatan Kerja

1. Selalu ikuti instruksi praktikum dari dosen, serta diskusikan jika terjadi *error* atau kendala dalam penggunaan simulator.
2. Hindari penggunaan perangkat komputer dan Unity secara berlebihan yang dapat menyebabkan kelelahan mata.
3. Gunakan perangkat komputer dengan posisi ergonomis, tinggi layar sejajar dengan pandangan dan posisi duduk yang baik untuk menghindari kelelahan saat melakukan simulasi.
4. Hindari membuka terlalu banyak aplikasi secara bersamaan agar Unity berjalan lancar dan mencegah komputer mengalami *crash*.

F. Langkah Kerja

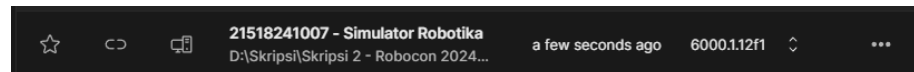
1. Buka *software* Unity Hub.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

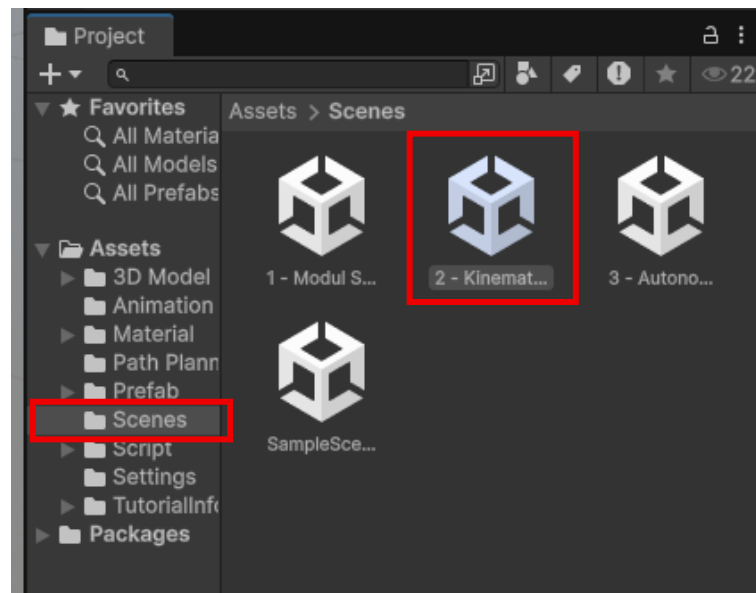
	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit



- Buka project simulator robotika.



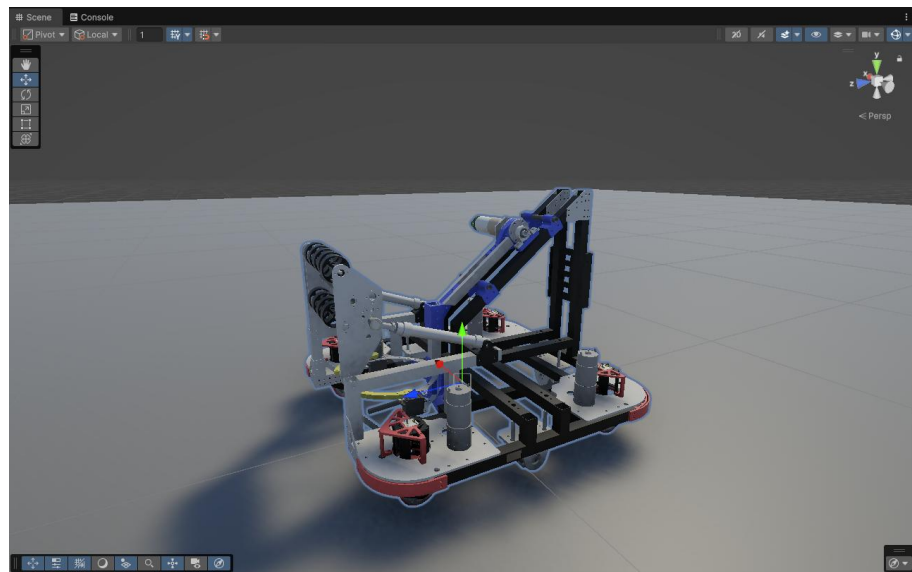
- Pada bagian Project buka folder Scenes pada Assets. Kemudian klik dua kali pada pembelajaran kedua dengan judul “2 - Kinematika Mobile Robot Swerve Drive”.



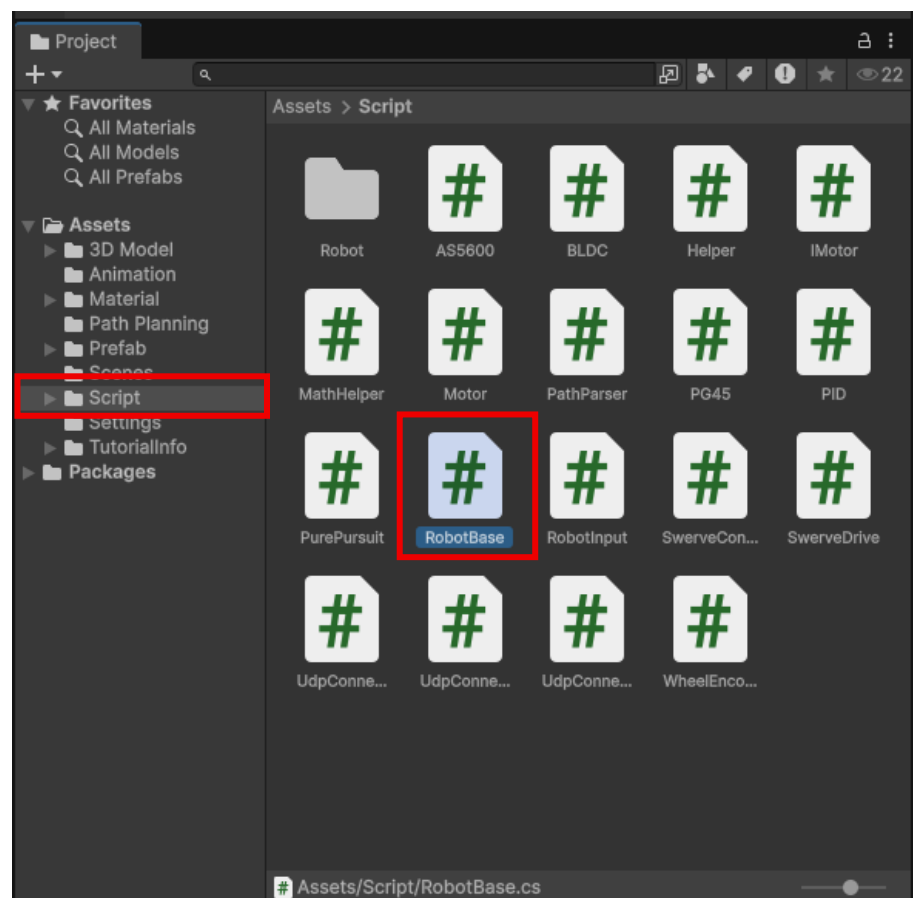
- Klik dua kali pada bagian Hierarchy lalu pada *game object* Robot Base Full. Secara otomatis, tampilan scene akan terfokus pada robot.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit



5. Buka script RobotBase.cs pada folder Scripts.



6. Selanjutnya buatlah method atau fungsi yang berada di dalam class RobotBase bernama SwerveDriveMove yang menerima empat

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

parameter yaitu: headingAngle, dforward, dstrafe, dan drotation dengan tipe data double dan mengembalikan nilai void.

```
public void SwerveDriveMove(double headingAngle, double
dforward, double dstrafe, double drotation)
```

7. Ketik kode untuk mentransformasi *input* kecepatan ke pergerakan kerangka robot. Kode ini berada dalam fungsi SwerveDriveMove yang sebelumnya telah dibuat.

```
double temp = dforward * Math.Cos(Mathf.Deg2Rad *
headingAngle) + dstrafe * Math.Sin(Mathf.Deg2Rad *
headingAngle);

double strafe = -dforward * Math.Sin(Mathf.Deg2Rad *
headingAngle) + dstrafe * Math.Cos(Mathf.Deg2Rad *
headingAngle);

double forward = temp;
strafe = -strafe;
double rotation = Math.Abs(drotation - (-0.0)) < 0.1f ?
0.0 : drotation;
```

Kode di atas ini berfungsi untuk:

- a. Mentransformasi vektor kecepatan dari kerangka global ke kerangka robot.
- b. *Heading* digunakan sebagai acuan untuk mengubah arah gerak agar sesuai dengan orientasi robot saat ini.
- c. *Strafe* atau gerak menyamping dibalik arahnya karena konvensi koordinat kiri dan kanan.
- d. *Forward* merupakan gerakan maju dan mundur robot.
- e. *Rotation* merupakan gerakan rotasi kerangka robot pada porosnya, untuk menghilangkan *noise* kecil pada *input* drotation, jika mendekati nol maka dianggap nol.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

8. Selanjutnya untuk mendefinisikan parameter geometri robot gunakan variabel L untuk panjang robot dan W untuk lebar robot.

```
// Panjang dan lebar robot (dalam meter)
double L = 0.65;
double W = 0.65;
```

9. Kemudian buat nilai untuk menghitung jarak diagonal dari pusat robot ke masing-masing roda, hal ini diperlukan untuk menghitung rotasi tiap rodanya.

```
double R = Math.Sqrt(L * L + W * W);
```

10. Pada bagian ini diterapkan *inverse kinematics* untuk robot *Swerve Drive* empat roda. Rotasi robot diuraikan ke dalam komponen translasi lokal masing-masing roda. Nilai A-D digunakan untuk menghitung arah dan kecepatan roda.

```
double A = strafe - rotation * (L / R);
double B = strafe + rotation * (L / R);
double C = forward - rotation * (W / R);
double D = forward + rotation * (W / R);
```

11. Setiap roda memiliki kecepatan sesuai dengan vektor resultan antara translasi dan rotasi dikalikan dengan *baseSpeedRpm* agar sesuai dengan satuan motor (RPM).

```
swerveKiriDepan.SetSpeed((float) (Math.Sqrt(B * B + C * C) * baseSpeedRpm));
swerveKananDepan.SetSpeed((float) (Math.Sqrt(B * B + D * D) * baseSpeedRpm));
swerveKananBelakang.SetSpeed((float) (Math.Sqrt(A * A + D * D) * baseSpeedRpm));
swerveKiriBelakang.SetSpeed((float) (Math.Sqrt(A * A + C * C) * baseSpeedRpm));
```

12. Langkah selanjutnya, untuk menghitung dan mengatur sudut *steering* roda pada robot dapat dihitung menggunakan fungsi **atan2** dari vektor(x,y). Fungsi **Mathf.Repeat** digunakan untuk menjaga agar sudut tetap berada dalam rentang $0^\circ - 360^\circ$.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```
float t1 = Mathf.Repeat((float) (Mathf.Rad2Deg *
Math.Atan2(B, C)), 360f); // Kiri Depan
float t2 = Mathf.Repeat((float) (Mathf.Rad2Deg *
Math.Atan2(B, D)), 360f); // Kanan Depan
float t3 = Mathf.Repeat((float) (Mathf.Rad2Deg *
Math.Atan2(A, D)), 360f); // Kanan Belakang
float t4 = Mathf.Repeat((float) (Mathf.Rad2Deg *
Math.Atan2(A, C)), 360f); // Kiri Belakang
```

13. Pada saat tidak terdapat input gerak, maka sudut *steering* roda tidak perlu diubah orientasinya. Ini menghemat energi dan menjaga posisi mekanik pada robot sesungguhnya.

```
// Jika tidak bergerak, biarkan sudut tetap
if (forward == 0.0f && strafe == 0.0f && rotation ==
0.0f)
{
    swerveKananDepan.SetAngle(swerveKananDepan.GetAngle());
    swerveKananBelakang.SetAngle(swerveKananBelakang.GetAngle());
    swerveKiriBelakang.SetAngle(swerveKiriBelakang.GetAngle());
    swerveKiriDepan.SetAngle(swerveKiriDepan.GetAngle());
}
else
{
    swerveKananDepan.SetAngle(t1);
    swerveKananBelakang.SetAngle(t2);
    swerveKiriBelakang.SetAngle(t3);
    swerveKiriDepan.SetAngle(t4);
}
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

14. Sebelum menerapkan pergerakan roda ke kerangka robot, selanjutnya buatlah fungsi bernama `ApplyForwardKinematic` yang tidak memiliki parameter dan mengembalikan nilai void.

```
public void ApplyForwardKinematic()
```

15. Fungsi `ApplyForwardKinematic` digunakan untuk menghitung dan menerapkan gerakan translasi dan rotasi robot berdasarkan informasi sudut dan kecepatan dari keempat roda *swerve*. Dalam kinematika maju (*forward kinematics*), informasi dari setiap aktuator (roda) digunakan untuk memperkirakan posisi dan orientasi robot terhadap lingkungan.

16. Hitung kecepatan pada semua roda di sumbu X dan Y lalu masukkan pada variabel dengan tipe data double. Pada sumbu X ambil kecepatan roda dan kalikan dengan cosinus dari sudut arah rodanya untuk menghasilkan komponen kecepatan pada sumbu X. Sedangkan, pada sumbu Y gunakan sinus.

```
// Komponen kecepatan pada sumbu X
double v1X = swerveKiriDepan.CalculateWheelSpeed() *
Math.Cos(Mathf.Deg2Rad * swerveKiriDepan.GetAngle());

double v2X = swerveKananDepan.CalculateWheelSpeed() *
Math.Cos(Mathf.Deg2Rad * swerveKananDepan.GetAngle());

double v3X = swerveKananBelakang.CalculateWheelSpeed()
*
Math.Cos(Mathf.Deg2Rad *
swerveKananBelakang.GetAngle());

double v4X = swerveKiriBelakang.CalculateWheelSpeed() *
Math.Cos(Mathf.Deg2Rad *
swerveKiriBelakang.GetAngle());

// Komponen kecepatan pada sumbu Y
double v1Y = swerveKiriDepan.CalculateWheelSpeed() *
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```
Math.Sin(Mathf.Deg2Rad * swerveKiriDepan.GetAngle());

double v2Y = swerveKananDepan.CalculateWheelSpeed() *
Math.Sin(Mathf.Deg2Rad * swerveKananDepan.GetAngle());
double v3Y = swerveKananBelakang.CalculateWheelSpeed()
*
Math.Sin(Mathf.Deg2Rad *
swerveKananBelakang.GetAngle());

double v4Y = swerveKiriBelakang.CalculateWheelSpeed() *
Math.Sin(Mathf.Deg2Rad *
swerveKiriBelakang.GetAngle());
```

17. Hitung rata-rata kecepatan translasi pada robot di sumbu X dan Y. Kalikan dengan Time.deltaTime untuk mendapatkan perpindahan dalam satu *frame* waktu.

```
double vxTotal = ((v1X + v2X + v3X + v4X) / 4) *
Time.deltaTime;
double vyTotal = ((v1Y + v2Y + v3Y + v4Y) / 4) *
Time.deltaTime;
```

18. Selanjutnya buat variabel R dengan *value* konstan 0.3, lalu cari nilai omega (ω) yang merupakan kecepatan sudut (rad/s) yang dihitung berdasarkan kombinasi kecepatan roda, persamaan ini mengasumsikan bahwa roda-roda terletak pada persegi dengan pusat rotasi di tengah robot.

```
const double R = 0.3;
double omega = (v1X + v1Y + v2X - v2Y - v3X - v3Y - v4X
+ v4Y) / (8 * R);
```

19. Sehingga didapatkan persamaan umum forward kinematic untuk translasi:

$$V_x = \frac{1}{4} \sum_{i=1}^4 v_i \cdot \cos(\theta_i)$$

$$V_y = \frac{1}{4} \sum_{i=1}^4 v_i \cdot \sin(\theta_i)$$

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Rotasi (ω) disusun dengan memperhitungkan torsi dari masing-masing roda relatif terhadap titik pusat robot.

20. Aplikasikan perhitungan pergerakan yang telah dilakukan untuk menggerakkan posisi dan orientasi kerangka robot secara virtual pada simulator berdasarkan hasil perhitungan (v_{xTotal} , v_{yTotal} , dan rotasi).

```
robotBase.transform.Translate(new
Vector3((float)vyTotal, 0f, -(float)vxTotal));

robotBase.transform.Rotate(Vector3.up, (float)(omega *
Mathf.Rad2Deg) * Time.deltaTime);
```

21. Panggil fungsi `ApplyForwardKinematic` di dalam fungsi `SwerveDriveMove` untuk menggerakkan posisi dan orientasi kerangka robot berdasarkan input yang diberikan pada *inverse kinematics*.

```
ApplyForwardKinematic();
```

22. Selanjutnya buat fungsi `UpdateOdometry` untuk menghitung posisi robot (X, Y) secara relatif terhadap posisi awalnya berdasarkan data encoder dan heading. Fungsi ini mengimplementasikan prinsip dasar odometri diferensial dengan tambahan *encoder* tengah untuk mendeteksi pergerakan lateral (*samping*) yang umum digunakan pada robot.

```
private void UpdateOdometry(long pulseFront, long
pulseRear, long pulseCenter, double heading)
```

23. Odometri adalah metode untuk memperkirakan posisi dan orientasi robot berdasarkan integrasi dari gerakan-gerakan kecil yang dihitung dari sensor seperti *encoder* dan IMU. Buat variabel konstan untuk menghitung jarak per *tick encoder* berdasarkan keliling roda dan PPR (*Pulse per revolution*), hal ini dikarenakan keliling roda dan PPR dari encoder dianggap sama.

```
double constant = 2 * Math.PI *
encoderDepan.GetWheelRadius() /
encoderDepan.GetEncoderPpr();
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

24. Setiap perubahan (*delta*) *pulse* dari masing-masing *encoder* dihitung sejak *update* sebelumnya.

```
long deltaFrontTicks = pulseFront - _lastLeftTicks;
long deltaRearTicks = pulseRear - _lastRightTicks;
long deltaCenterTicks = pulseCenter - _lastCenterTicks;
```

25. Ubah *heading* (orientasi) dari bentuk derajat ke radian agar dapat digunakan dalam fungsi trigonometri.

```
double robotHeading = heading * Mathf.Deg2Rad;
```

26. Hitung perpindahan robot dalam arah sumbu X (maju-mundur), sebagai rata-rata dari dua *encoder* depan dan belakang.

```
double displacementX = constant * (deltaFrontTicks +
deltaRearTicks) / 2.0;
```

27. Hitung perpindahan dalam arah Y (menyamping) dengan koreksi terhadap efek rotasi. Rasio 0.266 / 0.532 adalah kompensasi terhadap jarak roda terhadap jarak roda terhadap titik pusat rotasi robot.

```
double displacementY = constant * (deltaCenterTicks -
(deltaRearTicks - deltaFrontTicks) * 0.266f / 0.532f);
```

28. Rotasi koordinat lokal (*robot frame*) ke *global frame* berdasarkan *heading robot* menggunakan matriks rotasi 2D untuk menghitung *delta* perpindahan pada sumbu X dan Y.

```
double deltaX = displacementX * Math.Cos(robotHeading)
- displacementY * Math.Sin(robotHeading);
double deltaY = displacementX * Math.Sin(robotHeading)
+ displacementY * Math.Cos(robotHeading);
```

29. Simpan nilai *encoder* saat ini untuk digunakan sebagai acuan pada pemanggilan fungsi *update* berikutnya.

```
_lastLeftTicks = pulseFront;
_lastRightTicks = pulseRear;
_lastCenterTicks = pulseCenter;
```

30. Selanjutnya perbarui posisi odometri dalam koordinat global. Tanda minus pada *deltaX* dan *deltaY* dikarenakan konvensi sumbu yang terbalik pada Unity.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```
odometryPosition += new Vector2((float)-deltaX,
(float)-deltaY);
```

31. Mengirim data telemetri posisi dan heading ke program Python melalui UDP dan menampilkan data *encoder* mentah ke log di Unity.3

```
udpConnection.SendOdometryData(odometryPosition,
(float)robotHeading);
Debug.Log($"{pulseFront} {pulseRear} {pulseCenter}
{heading}");
```

32. Buatlah fungsi untuk reset posisi odometry ke (0,0).

```
private void ResetOdometry()
{
    odometryPosition = Vector2.zero;
}
```

33. Selanjutnya buat fungsi Update(). Fungsi ini merupakan bagian penting dari siklus utama dalam sistem kontrol robot, fungsi dijalankan setiap *frame* dan mengatur pengambilan input, kontrol *swerve*, manipulasi aktuator, dan pembaruan odometri.

```
private void Update()
```

34. Buat akses ke gamepad aktif menggunakan Unity Input System, namun jika tidak ada gunakan *input* dari keyboard.

```
var gamepad = Gamepad.current;
```

35. Buatlah variabel untuk menyimpan kondisi input.

```
float xAnalog;
float yAnalog;
bool l1;
bool r1;
bool l3 = false;
bool r3 = false;
float forward;
float strafe;
float rotation;
float gripperRotation = 0f;
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

36. Buatlah percabangan, apabila tidak terdapat gamepad yang terdeteksi maka gunakan *input* dari *keyboard* sebagai *input* utama. Jika menggunakan keyboard maka untuk maju dan mundur dapat menekan tombol W atau S, lalu untuk gerakan menyamping digunakan tombol A atau D, serta Q atau S untuk rotasi robot pada porosnya. Sedangkan jika menggunakan gamepad, untuk bergerak maju-mundur dan menyamping bisa menggunakan analog kiri, kemudian untuk rotasi bisa menggunakan L1 atau R1.

```

if (gamepad != null)
{
    xAnalog = gamepad.leftStick.x.value;
    yAnalog = gamepad.leftStick.y.value;

    l1 = gamepad.leftShoulder.isPressed;
    r1 = gamepad.rightShoulder.isPressed;
    l3 = gamepad.leftStickButton.isPressed;
    r3 = gamepad.rightStickButton.isPressed;

    forward = yAnalog;
    strafe = xAnalog;
    rotation = l1 ? 1 : r1 ? -1 : 0;

    gripperRotation = gamepad.rightStick.y.value;
}
else
{
    forward = Input.GetKey(KeyCode.W) ? 1.0f :
Input.GetKey(KeyCode.S) ? -1.0f : 0f;
    strafe = Input.GetKey(KeyCode.A) ? -1.0f :
Input.GetKey(KeyCode.D) ? 1.0f : 0f;
    rotation = Input.GetKey(KeyCode.Q) ? 1.0f :
Input.GetKey(KeyCode.E) ? -1.0f : 0f;
}

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

37. Gunakan tombol space pada keyboard digunakan untuk reset posisi odometry ke posisi awal.

```
if (Input.GetKeyDown(KeyCode.Space)) ResetOdometry();
```

38. Semua modul *swerve* akan menggunakan parameter PID yang sama setelah sebelumnya didapatkan parameter yang optimal. Buat instance baru bernama pid untuk digunakan di semua modul.

```
PID pid = new PID
{
    Kp = proportionalGain,
    Ki = intergralGain,
    Kd = derrivativeGain
};
```

39. Aplikasikan parameter PID yang telah dibuat ke semua *swerve*.

```
swerveKiriDepan.SetPidParameters(pid);
swerveKananDepan.SetPidParameters(pid);
swerveKananBelakang.SetPidParameters(pid);
swerveKiriBelakang.SetPidParameters(pid);
```

40. Buat *steering* roda pada *Swerve Drive* menggunakan kondisi di mana akan mencari jarak sudut terdekat ke titik target dari titik awal apabila diberikan boolean TRUE.

```
swerveKiriDepan.useNearestRotation =
this.useNearestRotation;
swerveKananDepan.useNearestRotation =
this.useNearestRotation;
swerveKananBelakang.useNearestRotation =
this.useNearestRotation;
swerveKiriBelakang.useNearestRotation =
this.useNearestRotation;
```

41. Normalisasi nilai *input* menjadi maksimal bernilai 1.

```
float maxVal = Math.Max(Math.Max(Math.Abs(forward),
Math.Abs(strafe)), Math.Abs(rotation));

if (maxVal > 1)
{
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

forward /= maxVal;
strafe /= maxVal;
rotation /= maxVal;
}

```

42. Panggil kinematika invers dengan memanggil fungsi `SwerveDriveMove` dengan mode orientasi lapangan atau orientasi robot melalui boolean yang dapat diset pada Unity bernama `fieldOriented`. Heading ditambah 90 derajat karena referensi sumbu utama robot berbeda dari koordinat Unity.

```

SwerveDriveMove((fieldOriented) ?
robotBase.transform.localEulerAngles.y + 90.0 : 90.0,
forward, strafe,
rotation);

```

43. Perbarui nilai odometry setiap robot bergerak, buat variabel sudutImu untuk menghitung rotasi robot di dunia simulasi Unity. Fungsi `UpdateOdometry()` dipanggil untuk memperbarui posisi robot di peta berdasarkan perubahan nilai *pulse encoder*.

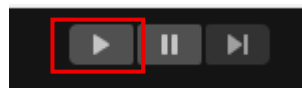
```

float sudutImu = Mathf.Repeat(-
robotBase.transform.localEulerAngles.y, 360f);

UpdateOdometry(encoderDepan.GetEncoderPulse(),
encoderBelakang.GetEncoderPulse(),
encoderKanan.GetEncoderPulse(), sudutImu);

```

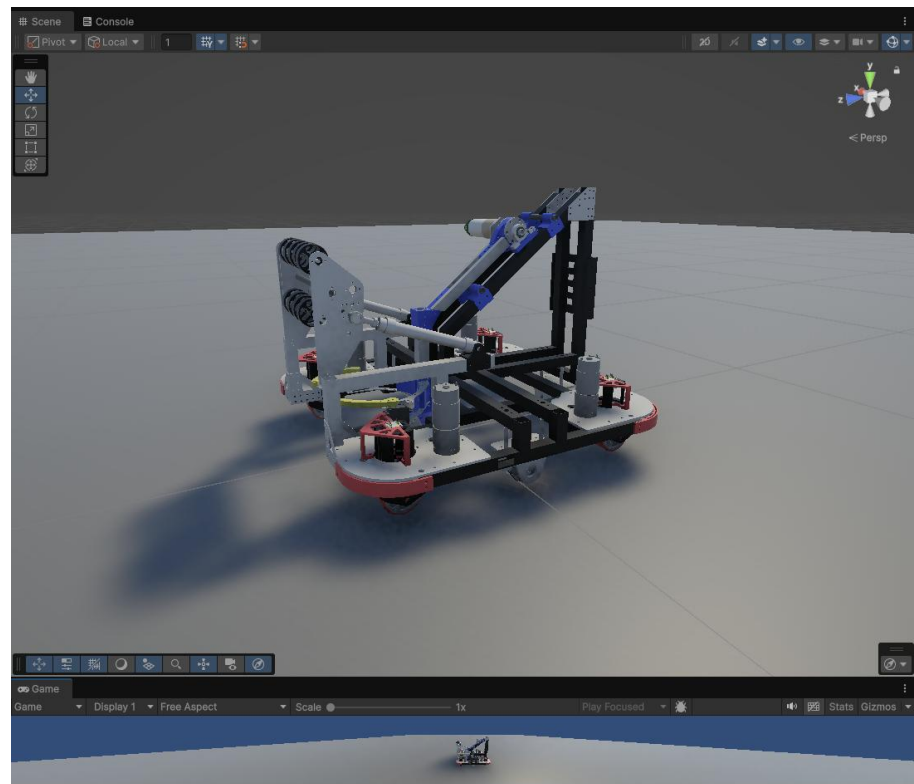
44. Kembali ke Unity Editor untuk mencoba pergerakan robot, gunakan play mode untuk menjalankan simulasi.



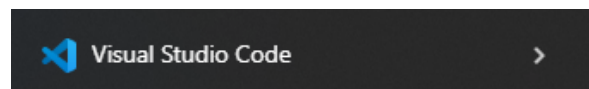
45. Klik sekali pada *window* Game, lalu gerakkan robot dengan tombol di keyboard WASD untuk bergerak atau QW untuk rotasi. Amati pergerakan pada *window* Scene.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

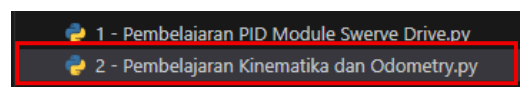
	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit



46. Buka VS Code untuk mengamati pergerakan robot menggunakan program Python, buka folder di mana LS1 disimpan.



47. Buatlah *file* dengan ekstensi .py atau program python baru dengan nama “2 - Pembelajaran Kinematika dan Odometry”.



48. Ketik kode program berikut ini.

```
import socket
import threading
import queue
import tkinter as tk
from tkinter import ttk, messagebox
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

from matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg, NavigationToolbar2Tk

# --- Konfigurasi UDP ---
UDP_IP = '127.0.0.1'
UDP_PORT = 5005

# --- Fungsi Listener UDP (Tidak ada perubahan) ---
def udp_listener_thread(q, sock, stop_event):
    """Mendengarkan data UDP dan memasukkannya ke dalam
    antrian."""
    print("UDP listener thread dimulai.")
    while not stop_event.is_set():
        try:
            sock.settimeout(1.0)
            data_bytes, _ = sock.recvfrom(1024)
            data_str = data_bytes.decode('utf-8')
            parts = data_str.split(',')
            if len(parts) == 3:
                x, y, angle = map(float, parts)
                q.put((x, y, angle))
        except socket.timeout:
            continue
        except (ValueError, IndexError):
            print(f"Menerima data tidak valid:
{data_str}")
        except Exception as e:
            if not stop_event.is_set():
                print(f"Error di listener UDP: {e}")
            break
    print("UDP listener thread dihentikan.")

class OdometryPlotterApp:

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

def __init__(self, master):
    self.master = master
    master.title("Real-time Odometry Plotter")
    master.geometry("900x900") # Sedikit lebih lebar
    untuk input baru

    # --- Inisialisasi data dan state ---
    self.x_path = []
    self.y_path = []
    self.current_x, self.current_y,
self.current_angle = 0.0, 0.0, 0.0

    # --- Setup Networking ---
    self.data_queue = queue.Queue()
    self.stop_event = threading.Event()
    self.sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
    self.sock.bind((UDP_IP, UDP_PORT))

    # <<< TAMBAHAN >>>: Variabel untuk batas plot
    statis
    self.x_min_var = tk.StringVar(value="-10.0")
    self.x_max_var = tk.StringVar(value="10.0")
    self.y_min_var = tk.StringVar(value="-10.0")
    self.y_max_var = tk.StringVar(value="10.0")

    self._create_widgets()
    self._start_listener()

    # Terapkan batas awal saat aplikasi dimulai
    self._apply_plot_limits()

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

        self._animation =
animation.FuncAnimation(self.fig, self._update_plot,
interval=100, blit=False)

        self.master.protocol("WM_DELETE_WINDOW",
self._on_closing)

    def _create_widgets(self):
        """Membuat semua elemen GUI."""
        # --- Frame Kontrol (Kiri) ---
        control_frame = ttk.Frame(self.master,
padding="10")
        control_frame.pack(side=tk.LEFT, fill=tk.Y,
padx=10, pady=10)

        ttk.Label(control_frame, text="Kontrol",
font=("Helvetica", 16, "bold")).pack(pady=(0, 10),
anchor='w')

        ttk.Button(control_frame, text="Reset Odometry
Data", command=self._reset_data).pack(fill=tk.X, pady=5,
ipady=5)

        # <<< TAMBAHAN >>>: Frame dan input untuk batas
plot
        limits_frame = ttk.LabelFrame(control_frame,
text="Batas Plot (meter)", padding="10")
        limits_frame.pack(fill=tk.X, pady=20)

        self._create_limit_entry(limits_frame, "X Min:",
self.x_min_var)
        self._create_limit_entry(limits_frame, "X Max:",
self.x_max_var)
        self._create_limit_entry(limits_frame, "Y Min:",
self.y_min_var)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

        self._create_limit_entry(limits_frame, "Y Max:",
self.y_max_var)

        ttk.Button(limits_frame, text="Apply Limits",
command=self._apply_plot_limits).pack(fill=tk.X,
pady=(10,0))

        self.status_label = ttk.Label(control_frame,
text="Menunggu data...", font=("Helvetica", 12),
wraplength=200)
        self.status_label.pack(side=tk.BOTTOM, pady=10)

        # --- Frame Plot (Kanan) ---
        plot_frame = ttk.Frame(self.master)
        plot_frame.pack(side=tk.RIGHT, fill=tk.BOTH,
expand=True, padx=(0, 10), pady=10)

        self.fig, self.ax = plt.subplots()
        self.ax.set_aspect('equal', adjustable='box')
        self.ax.set_title("Jejak Odometri Robot")
        self.ax.set_xlabel("Posisi X (meter)")
        self.ax.set_ylabel("Posisi Y (meter)")
        self.ax.grid(True, linestyle='--', alpha=0.6)

        self.path_line, = self.ax.plot([], [], 'b-',
label="Jejak Robot")
        self.robot_marker, = self.ax.plot([], [], 'ro',
markersize=8, label="Posisi Robot")
        self.ax.legend()

        self.canvas = FigureCanvasTkAgg(self.fig,
master=plot_frame)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

        toolbar = NavigationToolbar2Tk(self.canvas,
plot_frame, pack_toolbar=False)
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH,
expand=True)

    def _create_limit_entry(self, parent, label_text,
text_variable):
        """Helper untuk membuat entry batas."""
        frame = ttk.Frame(parent)
        frame.pack(fill=tk.X, pady=2)
        ttk.Label(frame, text=label_text,
width=7).pack(side=tk.LEFT)
        ttk.Entry(frame, textvariable=text_variable,
width=10).pack(side=tk.LEFT)

    def _start_listener(self):
        """Memulai thread listener UDP."""
        self.listener_thread = threading.Thread(
            target=udp_listener_thread,
            args=(self.data_queue, self.sock,
self.stop_event)
        )
        self.listener_thread.daemon = True
        self.listener_thread.start()

    def _update_plot(self, frame):
        """Fungsi yang dipanggil secara berkala untuk
memperbarui plot."""
        data_updated = False
        while not self.data_queue.empty():
            try:

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

        x, y, angle =
self.data_queue.get_nowait()
        self.x_path.append(x)
        self.y_path.append(y)
        self.current_x, self.current_y,
self.current_angle = x, y, angle
        data_updated = True
    except queue.Empty:
        break

    if data_updated:
        status_text = f"X: {self.current_x:.2f} | Y:
{self.current_y:.2f} | Sudut: {self.current_angle:.1f}°"
        self.status_label.config(text=status_text)

        # Update data di plot
        self.path_line.set_data(self.x_path,
self.y_path)
        self.robot_marker.set_data([self.current_x],
[self.current_y])

        # <<< PERBAIKAN >>>: Hapus rescaling otomatis
        # self.ax.relim()
        # self.ax.autoscale_view()

        # Gambar ulang canvas
        self.canvas.draw_idle()

def _apply_plot_limits(self):
    """Menerapkan batas plot dari input GUI."""
    try:
        x_min = float(self.x_min_var.get())
        x_max = float(self.x_max_var.get())

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

        y_min = float(self.y_min_var.get())
        y_max = float(self.y_max_var.get())

        if x_min >= x_max or y_min >= y_max:
            messagebox.showerror("Input Error",
"Nilai Min harus lebih kecil dari nilai Max.")
            return

        self.ax.set_xlim(x_min, x_max)
        self.ax.set_ylim(y_min, y_max)

        # Gambar ulang canvas untuk menerapkan batas
baru

        self.canvas.draw_idle()
        print(f"Batas plot diatur ke: X({x_min},
{x_max}), Y({y_min}, {y_max})")

    except ValueError:
        messagebox.showerror("Input Error", "Batas
plot harus berupa angka yang valid.")

    def _reset_data(self):
        """Menghapus semua data odometri dan membersihkan
plot."""
        print("Mereset data odometri...")
        self.x_path.clear()
        self.y_path.clear()

        self.path_line.set_data([], [])
        self.robot_marker.set_data([], [])

        # <<< PERBAIKAN >>>: Tidak perlu rescale, cukup
gambar ulang

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

```

self.canvas.draw_idle()

self.status_label.config(text="Data direset.
Menunggu data baru...")

def _on_closing(self):
    """Menangani penutupan aplikasi dengan bersih."""
    print("Menutup aplikasi...")
    self.stop_event.set()
    self.sock.close()
    if hasattr(self, 'listener_thread') and
self.listener_thread.is_alive():
        self.listener_thread.join(timeout=1.5)
        self.master.destroy()

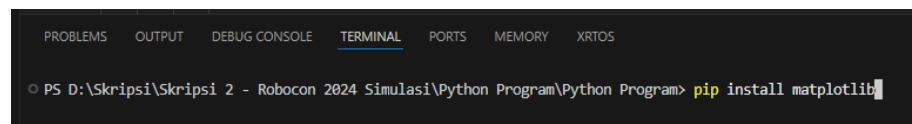
if __name__ == "__main__":
    root = tk.Tk()
    app = OdometryPlotterApp(root)
    root.mainloop()

```

49. Jika terdapat beberapa *library* yang belum dipasang pada sistem, maka *install* terlebih dahulu, berikut merupakan library eksternal yang harus diinstall.

Library	Sintaks Install
matplotlib	pip install matplotlib

50. Gunakan kombinasi keyboard (CTRL + `) untuk membuka terminal.



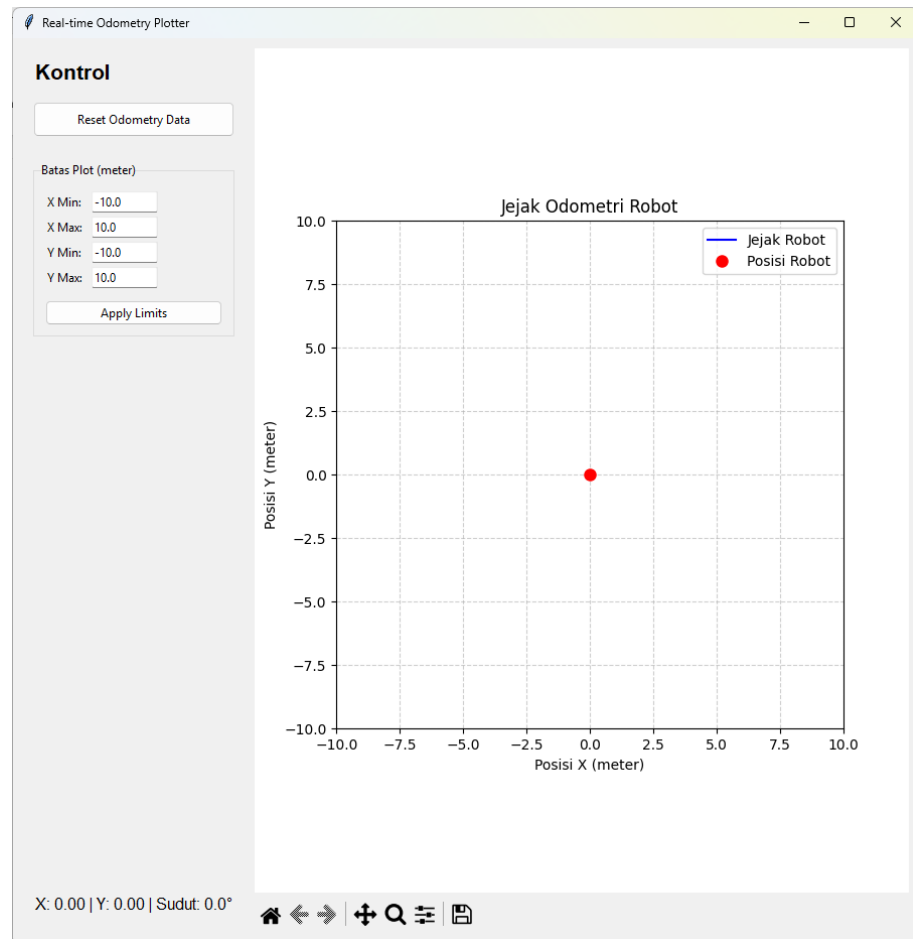
51. Jalankan kode python dengan sintaks (python ".\2 - Pembelajaran Kinematika dan Odometry.py") tanpa tanda kurung, atau gunakan tombol berikut pada VS Code.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit



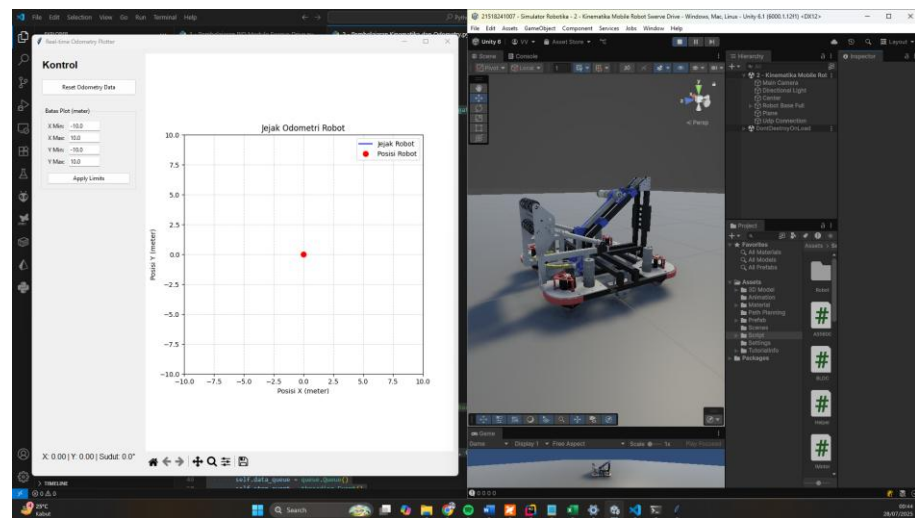
52. Berikut merupakan tampilan dari program Python yang telah berhasil dijalankan.



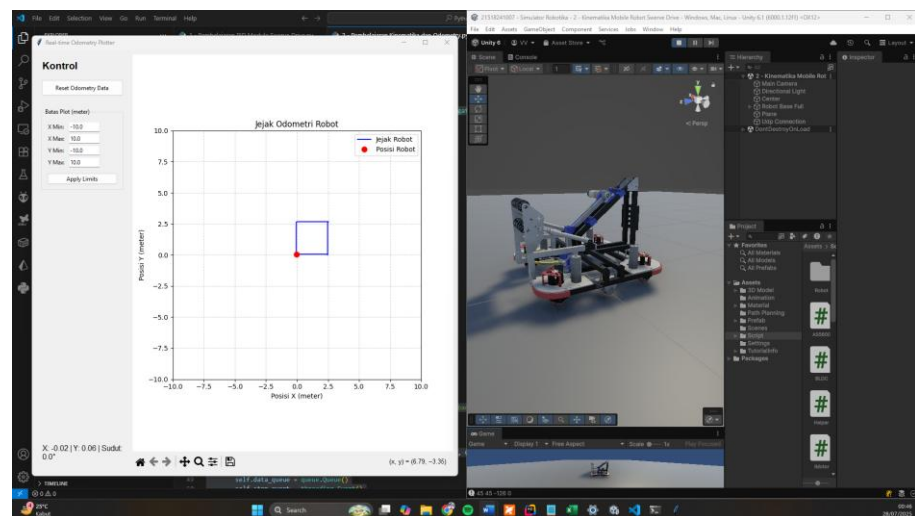
53. Buatlah *split screen* antara program Python dan Unity untuk mengamati secara langsung bagaimana arah gerak robot.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit



54. Coba menggerakkan robot ke arah maju – kanan – mundur – kiri untuk membentuk persegi dengan jarak masing-masing 2,5 meter seperti pada gambar di bawah ini.



55. Gunakan tombol “Reset Odometry Data” untuk menghapus *path* yang sebelumnya telah dilalui oleh robot, lalu coba ubah parameter batas plot untuk memberikan plot yang lebih besar atau lebih kecil tergantung pada kebutuhan.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Kontrol

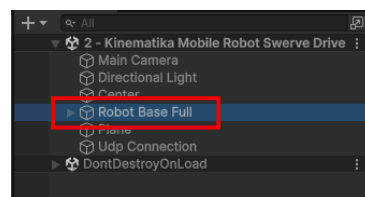
Reset Odometry Data

Batas Plot (meter)

X Min: -10.0
X Max: 10.0
Y Min: -10.0
Y Max: 10.0

Apply Limits

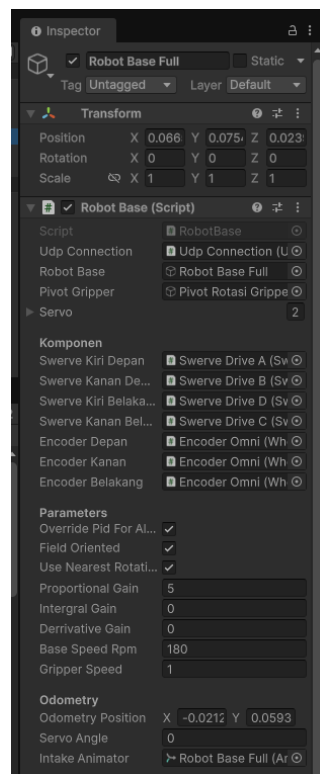
56. Untuk mengubah beberapa parameter pada robot dapat klik pada *game object* bernama “Robot Base Full” untuk mengubah parameter seperti kecepatan roda *base*, penggunaan jarak sudut terdekat untuk *steering swerve*, PID, mengamati lokasi odometri, dan mengubah apakah input berdasarkan orientasi lapangan atau berdasarkan orientasi robot.



57. Amati pada *window Inspector*.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit



G. Bahan Diskusi

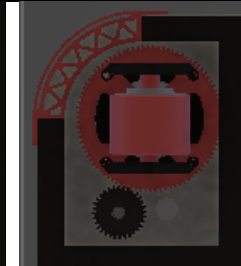
1. Apa perbedaan antara kinematika maju (forward kinematics) dan kinematika mundur (inverse kinematics) pada sistem Swerve Drive? Jelaskan bagaimana keduanya bekerja dalam simulasi ini.
2. Mengapa setiap roda pada *Swerve Drive* harus memiliki kemampuan berputar (*steering*) dan bergerak (*driving*)? Apa keunggulan desain ini dibanding sistem penggerak konvensional lainnya seperti *differential drive*?
3. Bagaimana ketelitian data *odometry*, seperti pembacaan *encoder* dan sudut orientasi, memengaruhi akurasi estimasi posisi robot pada sistem navigasi, dan bagaimana cara mengurangi kesalahan akumulatif dari *odometry* seiring waktu?
4. Dengan mengacu pada sistem koordinat kartesius, bagaimana arah gerak dan sudut masing-masing roda swerve drive berubah ketika robot bergerak ke berbagai arah. Ikuti instruksi pada tabel di bawah ini dan isi

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

data sesuai dengan keadaan pada simulasi. Sebagai catatan, arah kiri pada robot merupakan tempat di mana mekanisme *gripper* berada.

Tabel 6. Pengamatan Arah Gerak Roda Input Maju

Pengamatan Gerak Roda Input Maju			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang
			
Roda bergerak ke arah 90° pada koordinat kartesius	Roda bergerak ke arah 90° pada koordinat kartesius	Roda bergerak ke arah 90° pada koordinat kartesius	Roda bergerak ke arah 90° pada koordinat kartesius

Tabel 7. Pengamatan Arah Gerak Mundur

Pengamatan Gerak Roda Input Mundur			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

--	--	--	--

Tabel 8. Pengamatan Arah Gerak Samping Kanan

Pengamatan Gerak Roda Input Kanan			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Tabel 9. Pengamatan Arah Gerak Samping Kiri

Pengamatan Gerak Roda Input Kiri			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Tabel 10. Pengamatan Arah Gerak Rotasi *Clockwise*

Pengamatan Gerak Roda Input <i>Clockwise</i>			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Tabel 11. Pengamatan Arah Gerak Rotasi *Counter-clockwise*

Pengamatan Gerak Roda Input <i>Counter-clockwise</i>
--

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Tabel 12. Pengamatan Arah Gerak Kombinasi Maju-Kanan

Pengamatan Gerak Roda Input Kombinasi Maju-Kanan			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Tabel 13. Pengamatan Arah Gerak Kombinasi Maju-Kiri

Pengamatan Gerak Roda Input Kombinasi Maju-Kiri			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Tabel 14. Pengamatan Arah Gerak Kombinasi Mundur-Kiri

Pengamatan Gerak Roda Input Kombinasi Maju-Kiri			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

--	--	--	--

Tabel 15. Pengamatan Arah Gerak Kombinasi Mundur-Kanan

Pengamatan Gerak Roda Input Kombinasi Maju-Kiri			
Kiri Depan	Kanan Depan	Kiri Belakang	Kanan Belakang

5. Ikuti instruksi dan isilah tabel di bawah ini.

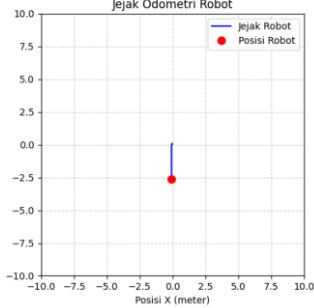
a. Gerak robot berorientasi lapangan

Tabel 16. Instruksi Gerakan Robot Berorientasi Lapangan

Gerak Robot Berorientasi Lapangan		
Gerakan	Hasil Gerakan	Penjelasan

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Tekan tombol S untuk bergerak sepanjang 2.5 meter		Robot mundur 2.5m dari posisi awal menuju posisi (0, -2.5)
Putar robot <i>clockwise</i> sebesar 90 derajat dengan tombol E		
Tekan tombol D untuk bergerak sepanjang 2.5 meter		
Putar robot <i>clockwise</i> sebesar 90 derajat dengan tombol E		
Tekan tombol W untuk bergerak sepanjang 2.5 meter		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Tekan tombol A untuk bergerak sepanjang 2.5 meter		
---	--	--

Jelaskan bagaimana posisi akhir robot dan simpulkan gerakan yang terjadi pada robot dengan mode orientasi lapangan.

.....

.....

.....

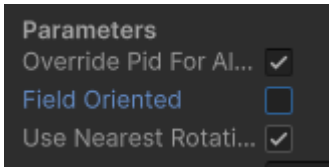
.....

.....

.....

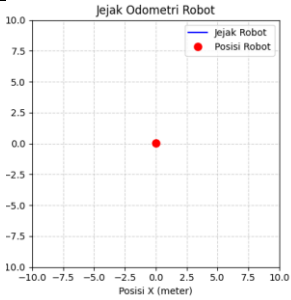
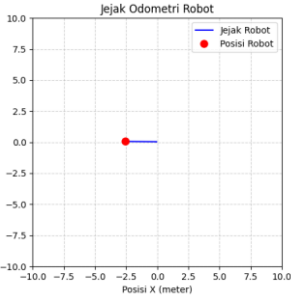
b. Gerak robot berorientasi robot

Tabel 17. Instruksi Gerakan Robot Berorientasi Robot

Gerak Robot Berorientasi Robot		
Gerakan	Hasil Gerakan	Penjelasan
Matikan field oriented		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Putar robot <i>counter-clockwise</i> sebesar 90 derajat dengan tombol Q		
Tekan tombol W hingga robot bergerak sebesar 2.5 meter		
Putar robot <i>clockwise</i> sebesar 90 derajat dengan tombol E		
Tekan tombol W hingga robot bergerak sebesar 2.5 meter		
Putar robot <i>counter-clockwise</i> sebesar 90 derajat dengan tombol Q		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 2	LS 2: Simulasi Kinematika dan Odometri <i>Swerve Drive Mobile Robot</i>	No. Job: 2
	Revisi: 00		4 x 50 menit

Tekan tombol S hingga robot bergerak sebesar 2.5 meter		
Tekan tombol A hingga robot bergerak sebesar 2.5 meter		

Jelaskan bagaimana posisi akhir robot dan simpulkan gerakan yang terjadi pada robot dengan mode orientasi robot.

.....

.....

.....

.....

.....

.....

- Buatlah rangkuman mengenai praktikum yang telah dilakukan.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi	No. Job: 3
	Revisi: 00	<i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	4 x 50 menit

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	No. Job: 3
	Revisi: 00		4 x 50 menit

A. Kompetensi

Mahasiswa mampu memahami, mengimplementasikan, dan menganalisis algoritma navigasi *autonomous mobile robot* berbasis *Pure Pursuit* melalui simulasi berbasis pemrograman.

B. Sub Kompetensi

Setelah melaksanakan praktikum, mahasiswa diharapkan mampu:

1. Menjelaskan prinsip kerja algoritma navigasi *Pure Pursuit* pada *mobile robot*.
2. Menerapkan algoritma *Pure Pursuit* dalam pemrograman untuk pelacakan lintasan (*path tracking*).
3. Memahami hubungan antara *lookahead point*, lintasan target, dan kontrol arah gerak robot.
4. Mengimplementasikan simulasi navigasi robot menggunakan algoritma *Pure Pursuit* pada Unity 3D.
5. Menganalisis performa navigasi robot berdasarkan parameter dan hasil lintasan yang diikuti.

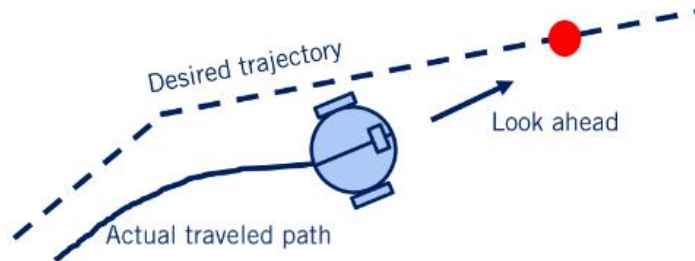
C. Dasar Teori

1. Pure Pursuit

Pure Pursuit merupakan sebuah algoritma *path following*. Algoritma ini bekerja dengan cara menghitung kecepatan angular (θ) yang menggerakkan robot dari posisi saat ini menuju ke sebuah titik yang dinamakan *look-ahead* yang berada di depan robot. Kecepatan linear robot diasumsikan konstan, sehingga dapat diubah kapanpun. Algoritma ini berusaha untuk mengejar *look-ahead* agar robot dapat mengikuti jalur.

Gambar 8 Algoritma Pure Pursuit
(Sumber: <https://medium.com>)

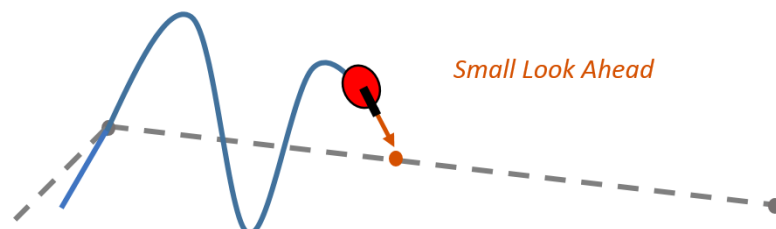
	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	No. Job: 3
	Revisi: 00		4 x 50 menit



Terdapat parameter yang dapat diubah yaitu seberapa jauh titik *look-ahead* tersebut. Input *waypoint* berupa koordinat dalam sumbu $[x, y]$, yang kemudian digunakan untuk menghitung kecepatan robot. *Pose* awal robot juga dimasukkan sebagai pengukuran dalam radian mulai dari 0° berlawanan dengan arah jarum jam. *Look Ahead distance* merupakan seberapa jauh pada *path* robot harus melihat ke depan untuk menghitung seberapa jauh titik *look-ahead* ditempatkan.

Nilai dari *look ahead* mempengaruhi pergerakan robot, apabila nilainya terlalu kecil akan membuat robot menjadi osilasi saat bergerak sesuai jalur, dan apabila terlalu besar akan membuat *path* yang dilalui robot menjadi terlalu menyimpang.

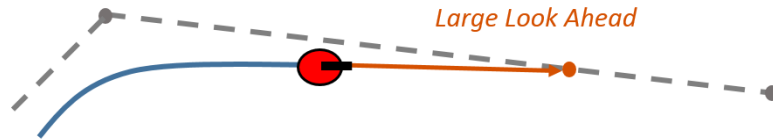
Gambar 9 Pengaruh Parameter Look Ahead Distance (a) kecil (b) besar



(a)

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit



(b)

D. Alat dan Bahan

1. Laptop atau komputer
2. Unity 3D
3. *Software* Visual Studio Code

E. Keselamatan Kerja

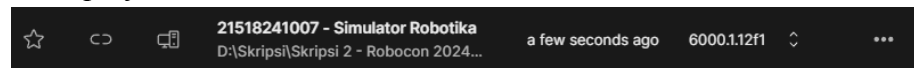
1. Selalu ikuti instruksi praktikum dari dosen, serta diskusikan jika terjadi *error* atau kendala dalam penggunaan simulator.
2. Hindari penggunaan perangkat komputer dan Unity secara berlebihan yang dapat menyebabkan kelelahan mata.
3. Gunakan perangkat komputer dengan posisi ergonomis, tinggi layar sejajar dengan pandangan dan posisi duduk yang baik untuk menghindari kelelahan saat melakukan simulasi.
4. Hindari membuka terlalu banyak aplikasi secara bersamaan agar Unity berjalan lancar dan mencegah komputer mengalami *crash*.

F. Langkah Kerja

1. Buka *software* Unity Hub.



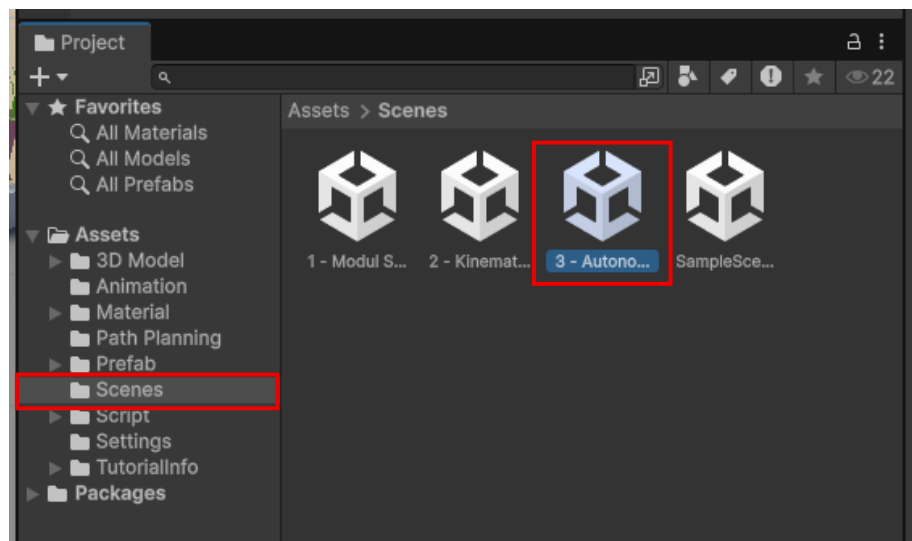
2. Buka *project* simulator robotika.



3. Pada bagian Project buka folder Scenes pada Assets. Kemudian klik dua kali pada pembelajaran ketiga dengan judul "3 - Autonomous Swerve Drive".

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit



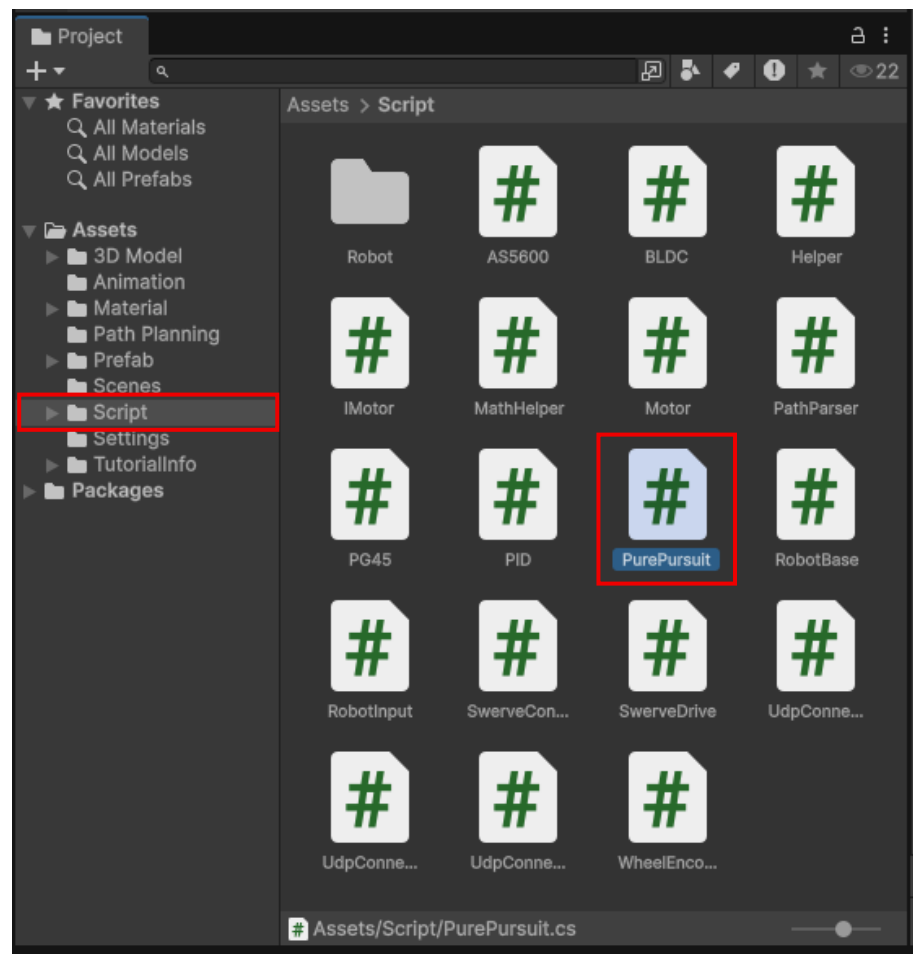
4. Klik dua kali pada Hierarchy lalu pada *game object* Robot Base Full. Maka secara otomatis, tampilan pada scene akan terfokus pada robot.



5. Buka script PurePursuit.cs pada folder Script di bagian Project Assets.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit



6. Buatlah class baru dengan nama PurePursuit yang memiliki *modifier* public.

```
public class PurePursuit
```

7. Buat beberapa variabel private untuk menyimpan indeks posisi lama, indeks posisi sekarang, dan parameter *Pure Pursuit* seperti gain dan look ahead point.

```
private int _oldNearestPointIndex;
private int _index;
private int _index2;
private double _k, _lfc, _lf, _velocity, _lfMax;
```

8. Buat fungsi untuk mencari nilai paling minimum pada sebuah array list dengan nama SearchMinValue yang memiliki parameter *input* yaitu List<double> dan mengembalikan data bertipe integer.

```
private int SearchMinValue(List<double> data)
{
    int index = 0;
    double minimumValue = data[0];

    for (int i = 0; i < data.Count; i++)
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	No. Job: 3
	Revisi: 00		4 x 50 menit

```

{
    if (data[i] < minimumValue)
    {
        minimumValue = data[i];
        index = i;
    }
}
return index;
}

```

9. Buat fungsi untuk mencari indeks jalur terdekat dengan posisi saat ini dengan nama `SearchTargetIndex` yang memiliki parameter *input* yaitu posisi x sekarang, posisi y sekarang, kecepatan, dan jalur itu sendiri yang mengembalikan nilai void.

```

private void SearchTargetIndex<T>(double xNow, double
yNow, double velocity, List<Vector2> path)

```

10. Pada fungsi `SearchTargetIndex` buat variabel untuk menyimpan jarak-jarak ke jalur dari posisi sekarang.

```

List<double> distance = new List<double>();

```

11. Jika nilai indeks merupakan nilai terakhir dari data jalur, maka gunakan nilai sebelumnya, dan kembalikan fungsi.

```

if (_index == path.Count)
{
    _oldNearestPointIndex = _index;
    return;
}

```

12. Jika nilai indeks terakhir bernilai -1, maka cari indeks jarak yang paling pendek dengan posisi robot saat ini. Hal ini dilakukan untuk mempercepat eksekusi pada pencarian jalur awal.

```

if (_oldNearestPointIndex == -1)
{
    for (int i = 0; i < path.Count; i++)
    {
        distance.Add(MathHelper.EuclideanDistance<double>(path[
i].x, path[i].y, xNow, yNow));
    }
    _index = SearchMinValue(distance);
    _oldNearestPointIndex = _index;
}

```

13. Buat percabangan apabila nilai indeks terakhir bukan -1, maka cari indeks path yang paling dekat dengan posisi robot saat ini dengan mempertimbangkan beberapa validasi indeks.

```

else
{
    _index = _oldNearestPointIndex;
    double distanceThisIndex =
MathHelper.EuclideanDistance<double>(path[_index].x,
path[_index].y, xNow, yNow);
}

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	No. Job: 3
	Revisi: 00		4 x 50 menit

```

while (true)
{
    double distanceNextIndex =
MathHelper.EuclideanDistance<double>(path[_index +
1].x, path[_index + 1].y, xNow, yNow);

    if (distanceThisIndex < distanceNextIndex)
break;

    if (_index + 1 < path.Count - 1) _index++;
    if (_index >= path.Count - 1)
    {
        _index = path.Count - 1;
        break;
    }
    distanceThisIndex = distanceNextIndex;
}
_oldNearestPointIndex = _index;
}

```

14. Hitung posisi *look ahead point* dengan mengkalikan variabel *gain* dengan kecepatan lalu ditambahkan dengan jarak minimum *look ahead point* yang dapat diatur pada Unity.

```
_lf = _k * velocity + _lfc;
```

15. Buat perulangan untuk mencari indeks jalur yang melebihi nilai *lf (look ahead point)*.

```

while (_lf >
MathHelper.EuclideanDistance<double>(path[_index].x,
path[_index].y, xNow, yNow))
{
    if (_index + 1 >= path.Count - 1) break;

    _index++;

    _oldNearestPointIndex = _index;
}

```

16. Perbarui nilai variabel indeks2 dengan indeks sekarang.

```
_index2 = _index;
```

17. Buat fungsi *public* yang dapat digunakan untuk memberikan kontrol Pure Pursuit dengan memanggilnya dari kendali utama, dengan nama *PurePursuitControl* yang memiliki input posisi saat ini, kecepatan, dan array list jalur dan mengembalikan nilai posisi target yang telah diproses.

```

public Vector2 PurePursuitControl<T>(double xNow,
double yNow, double velocity, List<Vector2> path)

```

18. Buat variabel berikut untuk menyimpan nilai target.

```

float targetx, targety;
Vector2 targetPoint = new Vector2();

```

19. Panggil fungsi *SearchTargetIndex* untuk mencari nilai indeks target.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	No. Job: 3
	Revisi: 00		4 x 50 menit

```
SearchTargetIndex<T>(xNow, yNow, velocity, path);
```

20. Buat validasi untuk nilai indeks apabila nilai indeks2 melebihi nilai indeks, maka perbarui nilai indeks menjadi sama dengan nilai indeks2.

```
if (_index2 >= _index)
{
    _index = _index2;
}
```

21. Validasi nilai indeks apakah kurang dari jumlah data jalur yang ada, apabila sesuai maka gunakan nilai indeks tersebut sebagai titik yang dituju, apabila tidak maka artinya nilai indeks melebihi data yang ada, maka kurangi dengan -1 untuk mengambil titik terakhir.

```
if (_index < path.Count)
{
    targetx = path[_index].x;
    targety = path[_index].y;
}
else
{
    _index = path.Count;
    targetx = path[path.Count - 1].x;
    targety = path[path.Count - 1].y;
}
```

22. Ubah nilai variabel TargetPoint sesuai dengan target yang sebelumnya telah diproses lalu kembalikan nilai titik target ke pemanggil.

```
targetPoint.x = targetx;
targetPoint.y = targety;

TargetPoint = targetPoint;

return targetPoint;
```

23. Selanjutnya masukan kode program berikut untuk inisialisasi dan mengatur parameter Pure Pursuit pada class PurePursuit.

```
public Vector2 TargetPoint { get; private set; }

public void SetIndex(int val)
{
    _index = val;
    _index2 = val;
    _oldNearestPointIndex = val;
}

public void SetK(double k)
{
    this._k = k;
}

public int GetIndex()
{
    return _index;
}
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

public void SetLfc(double val)
{
    _lf = val;
}

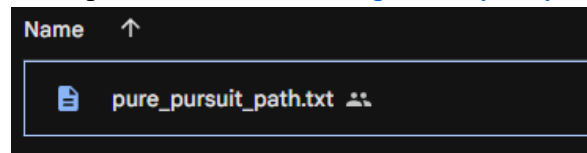
public void SetLfcMax(double val)
{
    _lfMax = val;
}

public void PurePursuitInitialize(double lf, double k)
{
    this._lfc = lf;
    this._k = k;
    _index = 0;
    _index2 = 0;
}

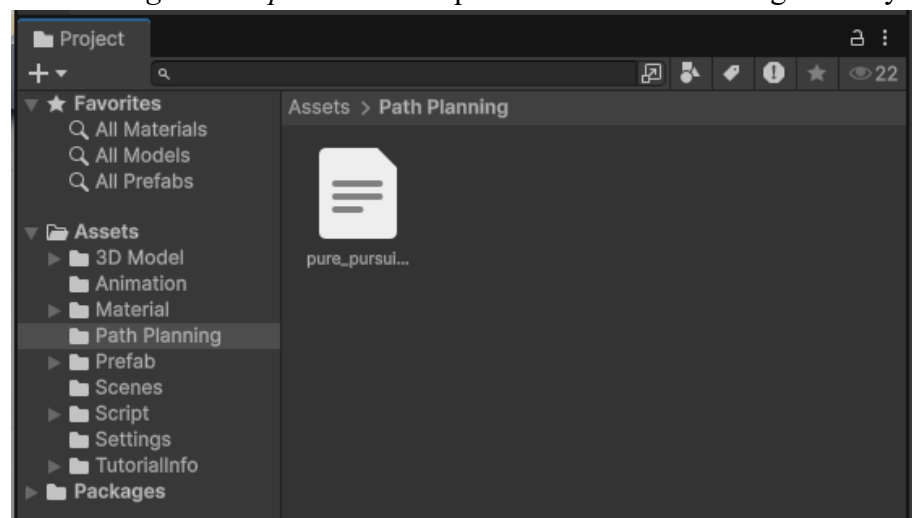
public void PurePursuitReset()
{
    _index2 = _index = _oldNearestPointIndex = 0;
    _lf = 0;
}

```

24. Unduh *path* atau jalur yang akan digunakan untuk navigasi otonom robot pada laman berikut <https://bit.ly/3UyS76k>.



25. Setelah file *path pure pursuit* selesai diunduh, langkah selanjutnya adalah *drag and drop* file tersebut pada folder Path Planning di Unity.



Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

26. Amati program pada script AutonomousRobot.cs pada fungsi OnEnable() di sini terdapat *directory* untuk file *path pure pursuit*. Ganti menjadi berikut:

```
string pathDirectory = Application.dataPath + @"/Path  
Planning/pure_pursuit_path.txt";
```

27. Amati pada fungsi Update().

28. Gunakan tombol *keyboard* R sebagai *reset* untuk mengembalikan posisi odometri ke titik awal yang telah ditentukan dan *reset* kembali *pure pursuit*.

```
if (Input.GetKeyDown(KeyCode.R))  
{  
    odometryPosition = zeroPosition;  
    _purePursuit.PurePursuitReset();  
    Debug.Log("Reset odometry position");  
}
```

29. Variabel untuk menyimpan nilai posisi odometri saat ini.

```
Vector2 now = odometryPosition;
```

30. Logika berikut digunakan untuk menaikkan posisi robot pada sumbu y di Unity agar robot tidak terlihat tenggelam dalam lapangan.

```
if (robotBase.transform.position.x > 2.0f)  
{  
    _yOffset = 0.097f + 0.098f;  
}  
else if (robotBase.transform.position.x > -1.995f)  
{  
    _yOffset = 0.097f;  
}  
else  
{  
    _yOffset = 0.0f;  
}
```

31. Gunakan tombol X pada keyboard untuk mengembalikan posisi kerangka robot pada posisi start seperti pada permainan ABU Robocon 2024.

```
if (Input.GetKey(KeyCode.X))  
{  
    robotBase.transform.position = new Vector3(-5.656f,  
0.122f, 5.504f);  
    robotBase.transform.rotation =  
Quaternion.Euler(0.0f, 0.0f, 0.0f);  
}
```

32. Gunakan tombol Space pada keyboard untuk melakukan navigasi *autonomous* apabila ditekan dan tahan dengan memanggil fungsi *SwerveAuto*.

```
if (Input.GetKey(KeyCode.Space))  
{  
    float velocity = Vector2.Distance(now,
```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	No. Job: 3
	Revisi: 00		4 x 50 menit

```

_lastPosition) * Time.deltaTime;

    Vector2 target =
    _purePursuit.PurePursuitControl<List<Vector2>>(now.x,
    now.y, velocity, _path);

    SwerveAuto(90.0f, transform.rotation.eulerAngles.y
    + 90.0f, target, now, baseSpeedRpm);
}

```

33. Pada fungsi SwerveAuto memiliki beberapa parameter yang harus diberikan, fungsi ini memastikan robot bergerak dengan mengubah input manual dengan input target posisi yang dituju dengan posisi sekarang.

```

public void SwerveAuto(float headingSet, float
headingNow, Vector2 target, Vector2 positionNow, float
baseSpeed)
{
    float forward = (target.y - positionNow.y);
    float strafe = (target.x - positionNow.x);

    float rotation = Mathf.DeltaAngle(headingSet,
headingNow) * 0.1f;

    // fmax(fmax(fabs(fwd), fabs(str)), fabs(rot));

    float maxVal =
    Mathf.Max(Mathf.Max(Mathf.Abs(forward),
    Mathf.Abs(strafe)), Mathf.Abs(rotation));

    if (maxVal > 1)
    {
        forward /= maxVal;
        strafe /= maxVal;
        rotation /= maxVal;
    }
    Debug.Log($"{forward} {strafe} {rotation}");

    SwerveDriveMove(baseSpeed, headingNow, -forward, -
    strafe, rotation);
}

```

34. Jika tidak menekan tombol Space, maka baca input manual seperti pada praktikum kedua.

```

else
{
    float forward = Input.GetKey(KeyCode.W) ? 1.0f :
    Input.GetKey(KeyCode.S) ? -1.0f : 0f;
    float strafe = Input.GetKey(KeyCode.A) ? -1.0f :
    Input.GetKey(KeyCode.D) ? 1.0f : 0f;
    float rotation = Input.GetKey(KeyCode.Q) ? 1.0f :
    Input.GetKey(KeyCode.E) ? -1.0f : 0f;

    if (Input.GetKeyDown(KeyCode.Space))

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

ResetOdometry();

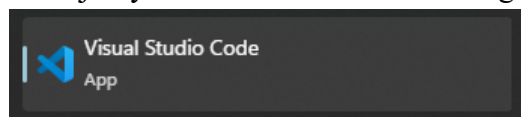
float maxVal = Math.Max(Math.Max(Math.Abs(forward),
Math.Abs(strafe)), Math.Abs(rotation));

if (maxVal > 1)
{
    forward /= maxVal;
    strafe /= maxVal;
    rotation /= maxVal;
}

SwerveDriveMove(baseSpeedRpm, (fieldOriented) ?
robotBase.transform.localEulerAngles.y + 90.0f : 90.0,
-forward, -strafe,
rotation);
}

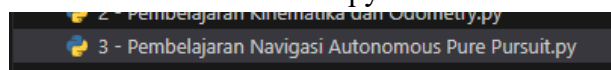
```

35. Selanjutnya buka VS Code untuk mengamati pergerakan dari robot.



36. Buka folder praktikum sebelumnya.

37. Buat file program python baru dengan judul “3 - Pembelajaran Navigasi Autonomous Pure Pursuit.py”.



38. Masukkan kode program berikut:

```

import socket
import threading
import queue
import tkinter as tk
from tkinter import ttk, messagebox
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.backends.backend_tkagg import
FigureCanvasTkAgg, NavigationToolbar2Tk
from PIL import Image, UnidentifiedImageError
from math import cos as _cos, sin as _sin, radians as
_rad
import pandas as pd

# --- KONFIGURASI APLIKASI ---
UDP_IP = '127.0.0.1'
UDP_PORT = 5005
TRACK_IMAGE_PATH = r"D:\Data
Kuliah\maestro2024\PathPlanning\lapangan_gui.png"

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi	No. Job: 3
	Revisi: 00	<i>Autonomous Mobile Robot dengan Pure Pursuit</i>	4 x 50 menit

```

REFERENCE_PATH_FILE = "path.csv"
TARGET_WIDTH = 12000
TARGET_HEIGHT = 6500

# --- FUNGSI LISTENER UDP ---
def udp_listener_thread(q, sock, stop_event):
    print("UDP listener thread dimulai.")
    while not stop_event.is_set():
        try:
            sock.settimeout(1.0)
            data_bytes, _ = sock.recvfrom(1024)
            data_str = data_bytes.decode('utf-8')
            parts = data_str.split(',')
            if len(parts) == 5:
                values = tuple(map(float, parts))
                q.put(values)
        except socket.timeout:
            continue
        except (ValueError, IndexError):
            print(f"Peringatan: Menerima data UDP tidak valid: {data_str}")
        except Exception as e:
            if not stop_event.is_set():
                print(f"Error kritis di listener UDP: {e}")
            break
    print("UDP listener thread dihentikan.")

# --- KELAS UTAMA APLIKASI GUI ---
class PurePursuitViewerApp:
    def __init__(self, master):
        self.master = master
        master.title("Real-time Pure Pursuit Viewer")
        master.geometry("900x950")

        self.robot_path_x, self.robot_path_y = [], []

        self.data_queue = queue.Queue()
        self.stop_event = threading.Event()
        self.sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        try:
            self.sock.bind((UDP_IP, UDP_PORT))

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

except OSError as e:
    messagebox.showerror("Socket Error", f"Gagal
bind ke port {UDP_PORT}.\n\nError: {e}")
    self.master.destroy()
    raise

self._create_widgets()
self._load_and_process_track_image(TRACK_IMAGE_PA
TH)

self._load_reference_path(REFERENCE_PATH_FILE)
self._start_listener()

self._animation =
animation.FuncAnimation(self.fig, self._update_plot,
interval=50, blit=True, init_func=self._init_plot)
self.master.protocol("WM_DELETE_WINDOW",
self._on_closing)

def _create_widgets(self):
    control_frame = ttk.Frame(self.master,
padding="10")
    control_frame.pack(side=tk.TOP, fill=tk.X)

    ttk.Button(control_frame, text="Reset Path",
command=self._reset_data).pack(side=tk.LEFT, padx=5,
ipady=3)

    self.status_label = ttk.Label(control_frame,
text="Menunggu data dari Unity...", font=("Helvetica",
10), wraplength=500)
    self.status_label.pack(side=tk.RIGHT, padx=10)

    plot_frame = ttk.Frame(self.master);
    plot_frame.pack(fill=tk.BOTH, expand=True, padx=10,
pady=10)

    self.fig, self.ax = plt.subplots();
    self.ax.set_aspect('equal', adjustable='box');
    self.ax.set_title("Visualisasi Pure Pursuit")
    self.ax.set_xlabel("Posisi X (meter)");
    self.ax.set_ylabel("Posisi Y (meter)");
    self.ax.grid(True, linestyle='--', alpha=0.6)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

        self.reference_path_line, = self.ax.plot([], [],
'k--', linewidth=1.5, label="Path Referensi", zorder=1)

        # <<< PERBAIKAN 1: Tambahkan animated=True untuk
        jejak robot >>>
        self.path_line, = self.ax.plot([], [], 'c-',
linewidth=2, label="Jejak Robot", zorder=2,
animated=True)

        self.target_marker, = self.ax.plot([], [], 'go',
markersize=10, label="Titik Target", zorder=4,
animated=True)
        self.lookahead_line, = self.ax.plot([], [], 'r--',
linewidth=1, label="Garis Lookahead", zorder=3,
animated=True)
        self.robot_pos_marker, = self.ax.plot([], [],
'ro', markersize=8, label="Robot", zorder=5,
animated=True)
        self.robot_orient_line, = self.ax.plot([], [],
'r-', linewidth=2, zorder=6, animated=True)

        self.ax.legend(loc='upper right')

        self.canvas = FigureCanvasTkAgg(self.fig,
master=plot_frame)
        toolbar = NavigationToolbar2Tk(self.canvas,
plot_frame, pack_toolbar=False);
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.canvas.get_tk_widget().pack(fill=tk.BOTH,
expand=True)

    def _load_and_process_track_image(self, filepath):
        try: original_img = Image.open(filepath)
        except (FileNotFoundError,
UnidentifiedImageError) as e: messagebox.showerror("Image
Load Error", f"Gagal memuat gambar
dari:\n{filepath}\n\nError: {e}"); return
        w, h = original_img.width, original_img.height;
        target_size = (TARGET_WIDTH, TARGET_HEIGHT) if w > h else
(TARGET_HEIGHT, TARGET_WIDTH)
        processed_img = original_img.resize(target_size,
Image.Resampling.LANCZOS).transpose(Image.FLIP_LEFT_RIGHT
).rotate(180)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

img_width_m, img_height_m = processed_img.width /
1000.0, processed_img.height / 1000.0
plot_extent = [0, img_width_m, 0, img_height_m]
self.ax.imshow(processed_img, extent=plot_extent,
aspect='equal', zorder=0, origin='lower')
self.ax.set_xlim(plot_extent[0], plot_extent[1]);
self.ax.set_ylim(plot_extent[2], plot_extent[3]);
self.canvas.draw_idle()

def _load_reference_path(self, filepath):
    try:
        path_df = pd.read_csv(filepath); ref_x =
path_df['x'].values; ref_y = path_df['y'].values
        self.reference_path_line.set_data(ref_x,
ref_y); self.canvas.draw_idle()
        print(f"Path referensi dimuat dari
'{filepath}'.")
    except FileNotFoundError: print(f"Peringatan:
File path referensi '{filepath}' tidak ditemukan.")
    except Exception as e: messagebox.showerror("Path
File Error", f"Gagal membaca file '{filepath}'.\nPastikan
formatnya benar.\n\nError: {e}")

def _start_listener(self):
    self.listener_thread =
threading.Thread(target=udp_listener_thread,
args=(self.data_queue, self.sock, self.stop_event))
    self.listener_thread.daemon = True;
self.listener_thread.start()

def _init_plot(self):
    """Fungsi inisialisasi untuk blitting.
Mengembalikan semua artis yang di-blit."""
    self.robot_pos_marker.set_data([], [])
    self.robot_orient_line.set_data([], [])
    self.target_marker.set_data([], [])
    self.lookahead_line.set_data([], [])
    self.path_line.set_data([], []) # <<< PERBAIKAN
2: Inisialisasi jejak di sini juga

    # Kembalikan semua elemen yang memiliki
animated=True

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

        return self.path_line, self.robot_pos_marker,
self.robot_orient_line, self.target_marker,
self.lookahead_line

    def _update_plot(self, frame):
        """Fungsi animasi yang dioptimalkan."""
        latest_data = None
        while not self.data_queue.empty():
            try:
                latest_data =
self.data_queue.get_nowait()
            except queue.Empty:
                break

        if latest_data:
            rx, ry, ra, tx, ty = latest_data

            self.robot_path_x.append(rx)
            self.robot_path_y.append(ry)
            self.path_line.set_data(self.robot_path_x,
self.robot_path_y)

            self._update_robot_marker(rx, ry, ra)
            self.target_marker.set_data([tx], [ty])
            self.lookahead_line.set_data([rx, tx], [ry,
ty])

            status_text = f"Robot: ({rx:.2f}, {ry:.2f},
{ra:.1f}°) | Target: ({tx:.2f}, {ty:.2f})"
            self.status_label.config(text=status_text)

        # <<< PERBAIKAN 3: Kembalikan jejak sebagai
bagian dari tuple blit >>>
        return self.path_line, self.robot_pos_marker,
self.robot_orient_line, self.target_marker,
self.lookahead_line

    def _update_robot_marker(self, x, y, angle_deg):
        self.robot_pos_marker.set_data([x], [y]); length
= 0.3; angle_rad = _rad(angle_deg)
        end_x = x + length * _cos(angle_rad); end_y = y +
length * _sin(angle_rad)

```

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

```

        self.robot_orient_line.set_data([x, end_x], [y,
end_y])

    def _reset_data(self):
        """Mereset jejak robot yang telah dilalui."""
        if messagebox.askokcancel("Konfirmasi Reset",
"Apakah Anda yakin ingin menghapus semua jejak robot?"):
            print("Mereset jejak robot...")
            self.robot_path_x.clear()
            self.robot_path_y.clear()
            # Tidak perlu memanggil set_data di sini
            karena _init_plot akan menanganinya
            # saat animasi me-reset.

    def _on_closing(self):
        if messagebox.askokcancel("Keluar", "Apakah Anda
yakin ingin keluar?"):
            self.stop_event.set(); self.sock.close()
            if hasattr(self, 'listener_thread') and
self.listener_thread.is_alive():
self.listener_thread.join(timeout=1.5)
            self.master.destroy()

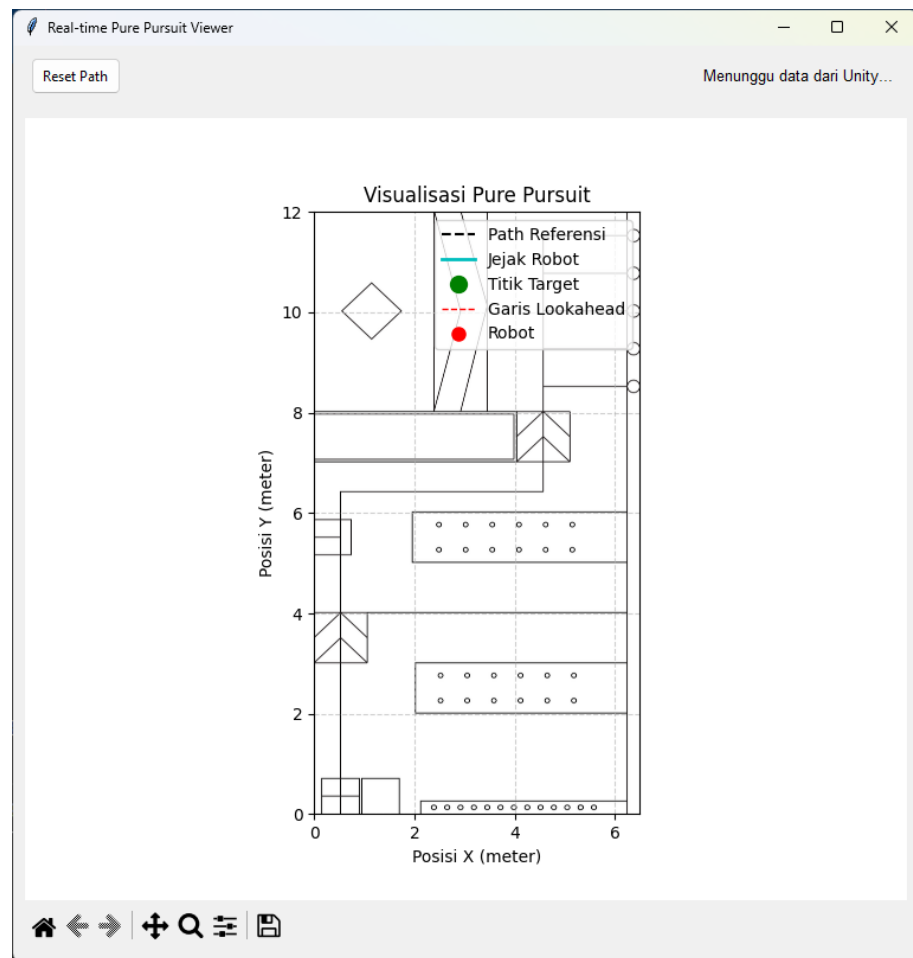
if __name__ == "__main__":
    try:
        root = tk.Tk()
        app = PurePursuitViewerApp(root)
        root.mainloop()
    except Exception as e:
        import traceback; messagebox.showerror("Fatal
Error", f"Aplikasi gagal dijalankan.\n\nError: {e}")
        print(f"Gagal menjalankan aplikasi: {e}");
        traceback.print_exc()

```

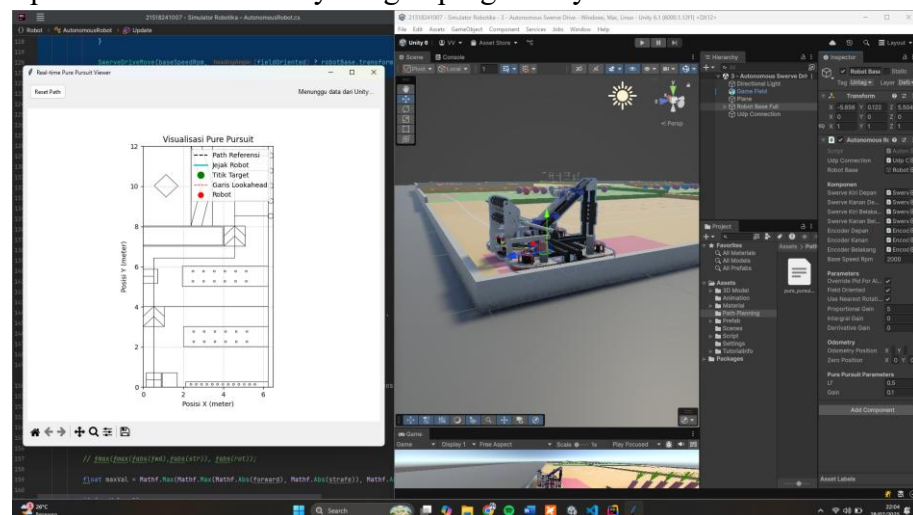
39. Jalankan program Python yang sebelumnya telah dibuat. Berikut adalah tampilan program yang telah berhasil dijalankan.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit



40. Split screen antara Unity dengan program Python.



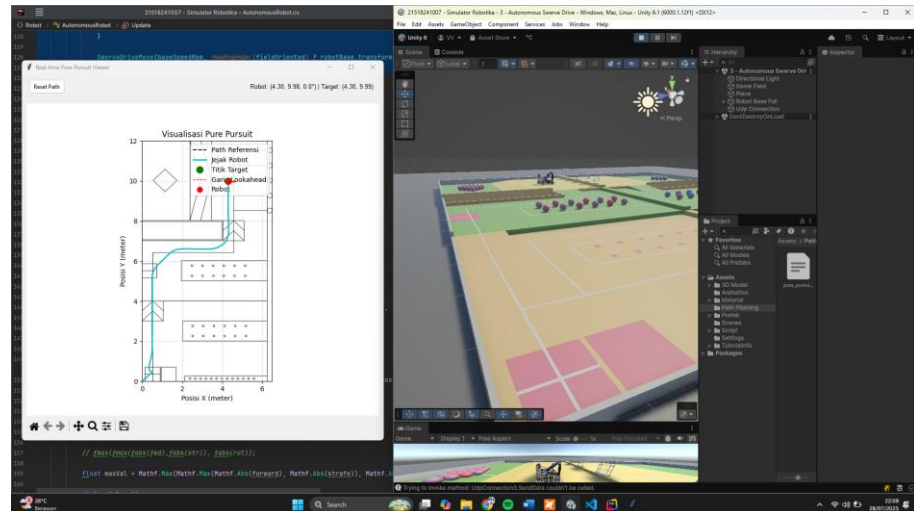
41. Jalankan simulasi pada Unity dengan menekan tombol berikut.



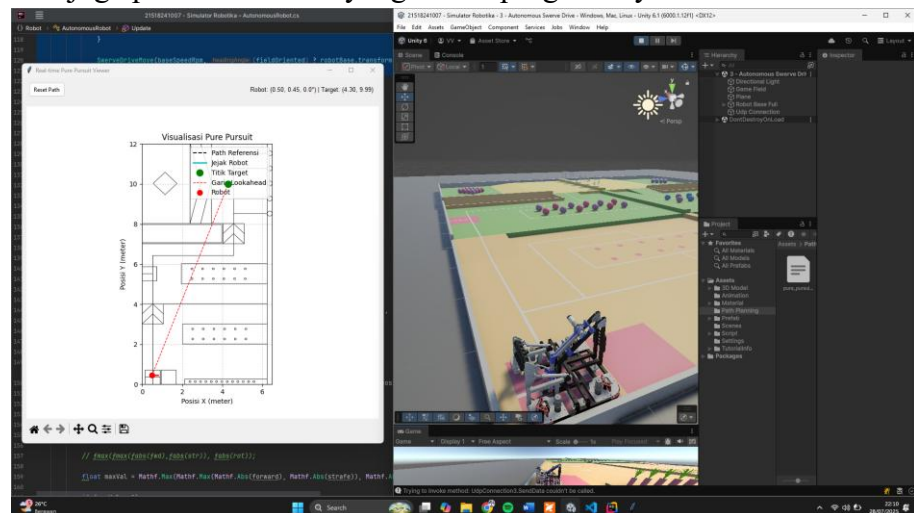
Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

42. Klik sekali pada *window* Game untuk mengaktifkan input Unity, selanjutnya tekan tombol R dan X pada keyboard untuk mengembalikan posisi robot ke *start* dan *reset* kembali posisi odometri serta *pure pursuit*.
43. Tekan dan tahan tombol Space pada *keyboard* untuk memulai simulasi, berhenti sampai robot berada di lantai 3.



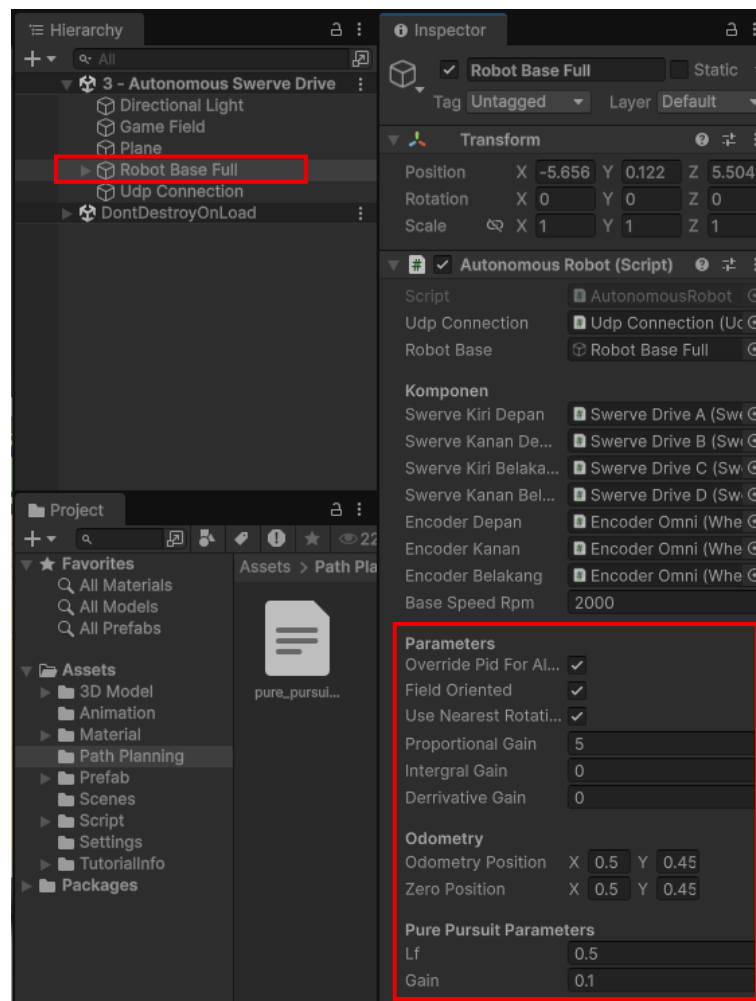
44. Kembalikan robot pada posisi awal lalu *reset* odometri dan *pure pursuit*, klik juga pada Reset Path yang ada di program Python.



45. Untuk mengubah parameter *pure pursuit*, klik pada *game object* bernama Robot Base Full di Hierarchy. Amati pada Inspector, terdapat parameter yang dapat diubah untuk odometri, PID, *pure pursuit*, dan swerve.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit



G. Bahan Diskusi

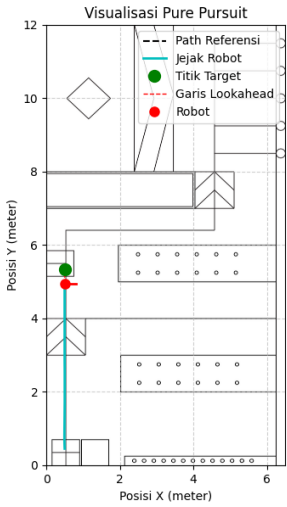
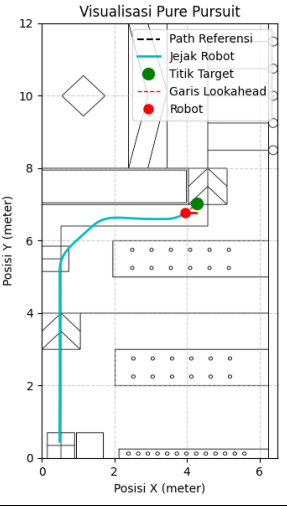
- Ubahlah parameter *pure pursuit* sesuai dengan instruksi pada tabel, kemudian jelaskan bagaimana respon gerakan robot pada lintasan yang telah ditentukan.

Tabel 18. Pengamatan Navigasi *Pure Pursuit* 1

Lf = 0.5 Gain = 0.1		
No.	Lintasan Gerak Robot	Penjelasan

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi	No. Job: 3
	Revisi: 00	<i>Autonomous Mobile Robot dengan Pure Pursuit</i>	4 x 50 menit

1		Robot bergerak lurus menuju lantai 2
2		Pada koordinat (0.55, 5.55) robot mulai bergerak menyamping ke arah kanan atas menuju target (1.62, 6.66) dengan sedikit overshoot pada sumbu Y
3		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

4		
dst		

Tabel 19. Pengamatan Navigasi *Pure Pursuit 2*

Lf = 2 Gain = 0.1		
No.	Lintasan Gerak Robot	Penjelasan
1		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi	No. Job: 3
	Revisi: 00	<i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	4 x 50 menit

2		
3		
4		
dst		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

Tabel 20. Pengamatan Navigasi *Pure Pursuit* 3

Lf = 10 Gain = 0.1		
No.	Lintasan Gerak Robot	Penjelasan
1		
2		
3		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi	No. Job: 3
	Revisi: 00	<i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	4 x 50 menit

4		
dst		

Tabel 21. Pengamatan Navigasi *Pure Pursuit* 3

Lf = 0.1 Gain = 0.1		
No.	Lintasan Gerak Robot	Penjelasan
1		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi	No. Job: 3
	Revisi: 00	<i>Autonomous Mobile Robot</i> dengan <i>Pure Pursuit</i>	4 x 50 menit

2		
3		
4		
dst		

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------

	DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO		
	LABSHEET PRAKTIK ROBOTIKA		
	Labsheet 3	LS 3: Simulasi Navigasi <i>Autonomous Mobile Robot</i>	No. Job: 3
	Revisi: 00	dengan <i>Pure Pursuit</i>	4 x 50 menit

- Jelaskan bagaimana pengaruh nilai *lookahead* point jika terlalu kecil atau terlalu besar. Bagaimana pengaruhnya terhadap akurasi dan lintasan yang dilewati oleh robot?
- Jelaskan keunggulan dan kelemahan, serta perbandingan algoritma *Pure Pursuit* dengan navigasi lain seperti *Stanley Controller*, DWA (*Dynamic Window Approach*), atau PID path tracking.
- Jelaskan bagaimana pengaruh kecepatan robot terhadap performa algoritma *Pure Pursuit*. Apa yang perlu disesuaikan saat kecepatan robot tinggi agar lintasan tetap dapat diikuti dengan baik?
- Apakah terjadi gejala osilasi (zigzag) atau gerakan tidak stabil saat robot mengikuti jalur? Jika ya, apa kemungkinan penyebabnya dan bagaimana cara mengurangnya?
- Buatlah rangkuman mengenai praktikum.

Disusun oleh: Vincent Kenutama P	Fakultas Teknik Universitas Negeri Yogyakarta	Diperiksa oleh:
-------------------------------------	--	-----------------